IS6400 Business Data Analytics

ZAHNG Yanyu

City University of Hong Kong

Lecture 11

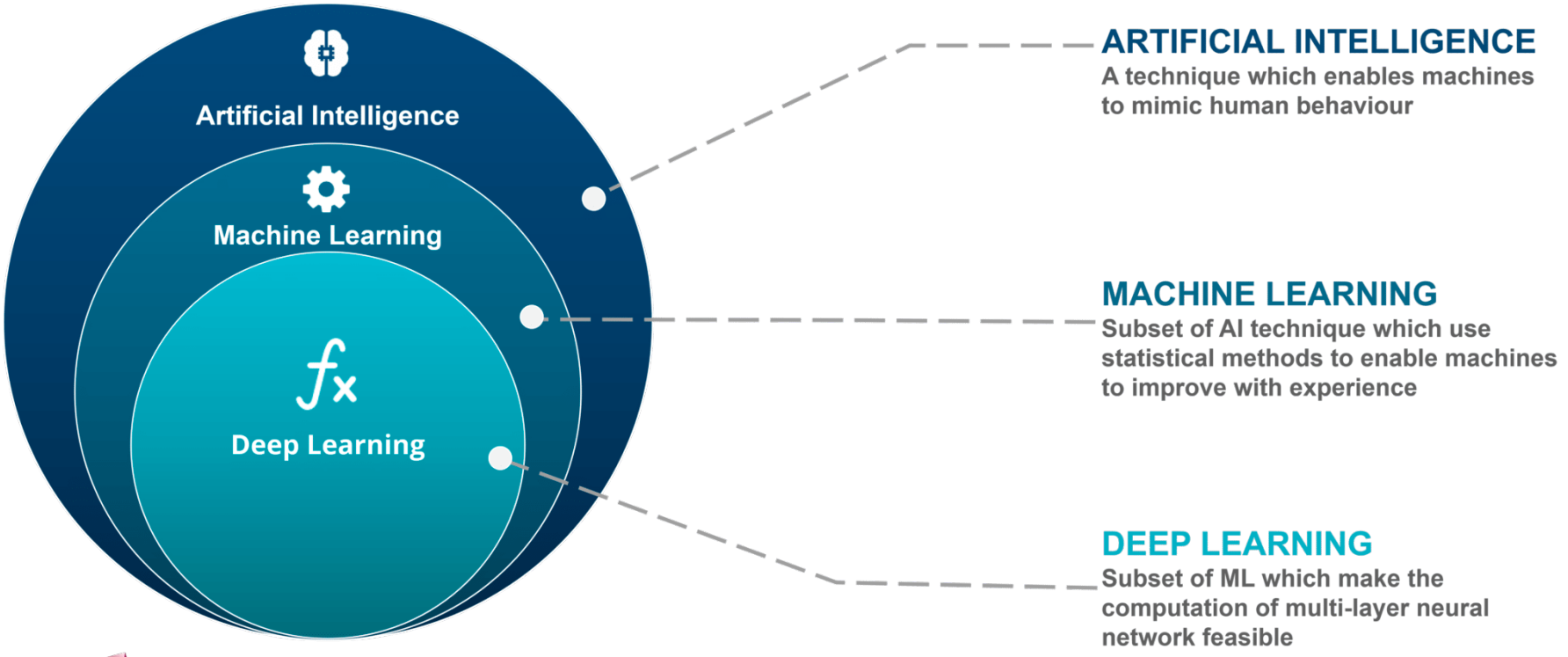
- Intro-Deep Learning
  - ANN
  - RNN, LSTM
  - NLP

Tutorial 11:

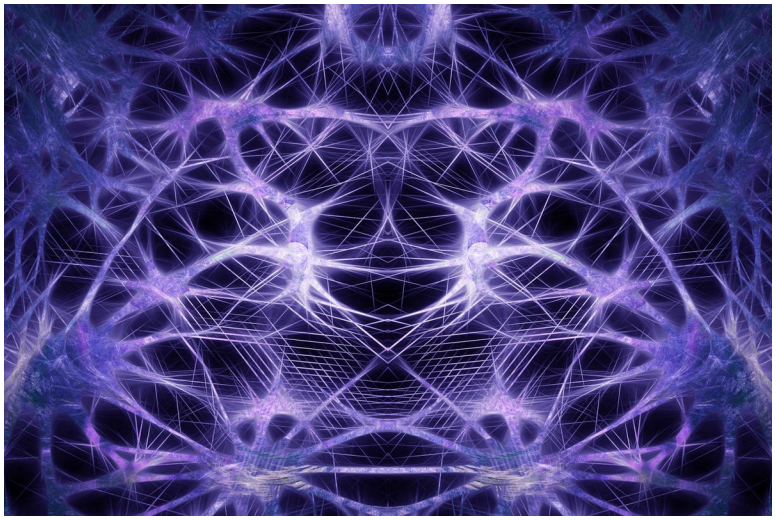
- Deep Learning

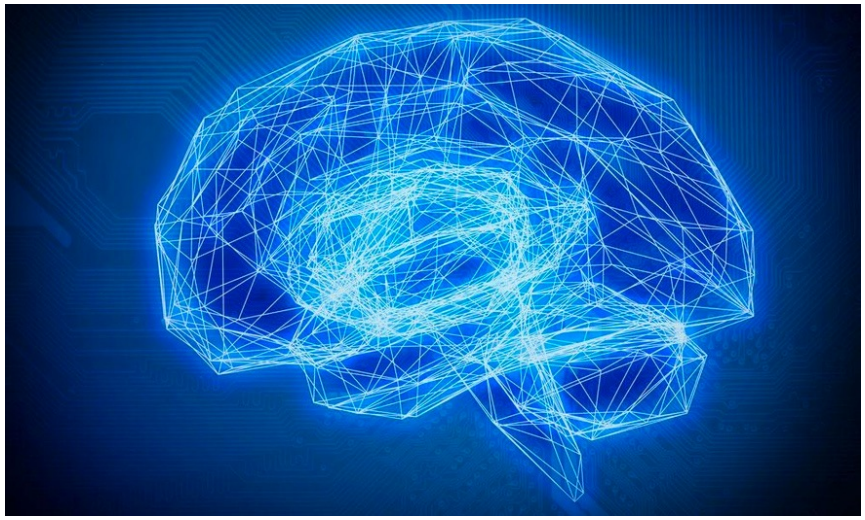
What is Deep Learning?

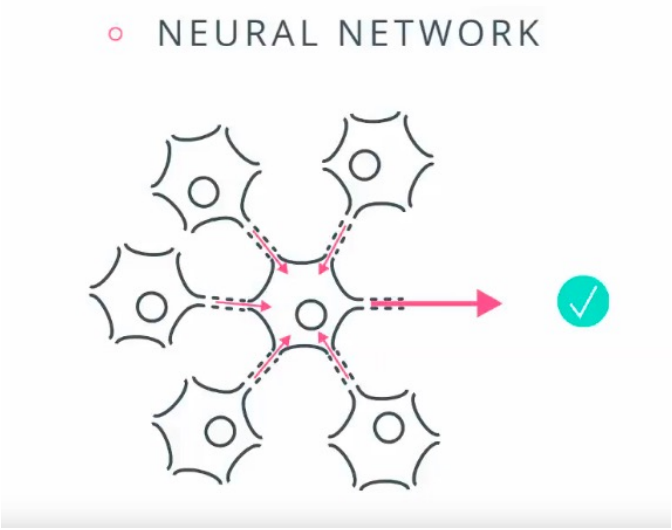
Deep learning (also known as **deep structured learning**) is part of a broader family of machine learning methods based on artificial neural networks.



Artificial Neural Networks







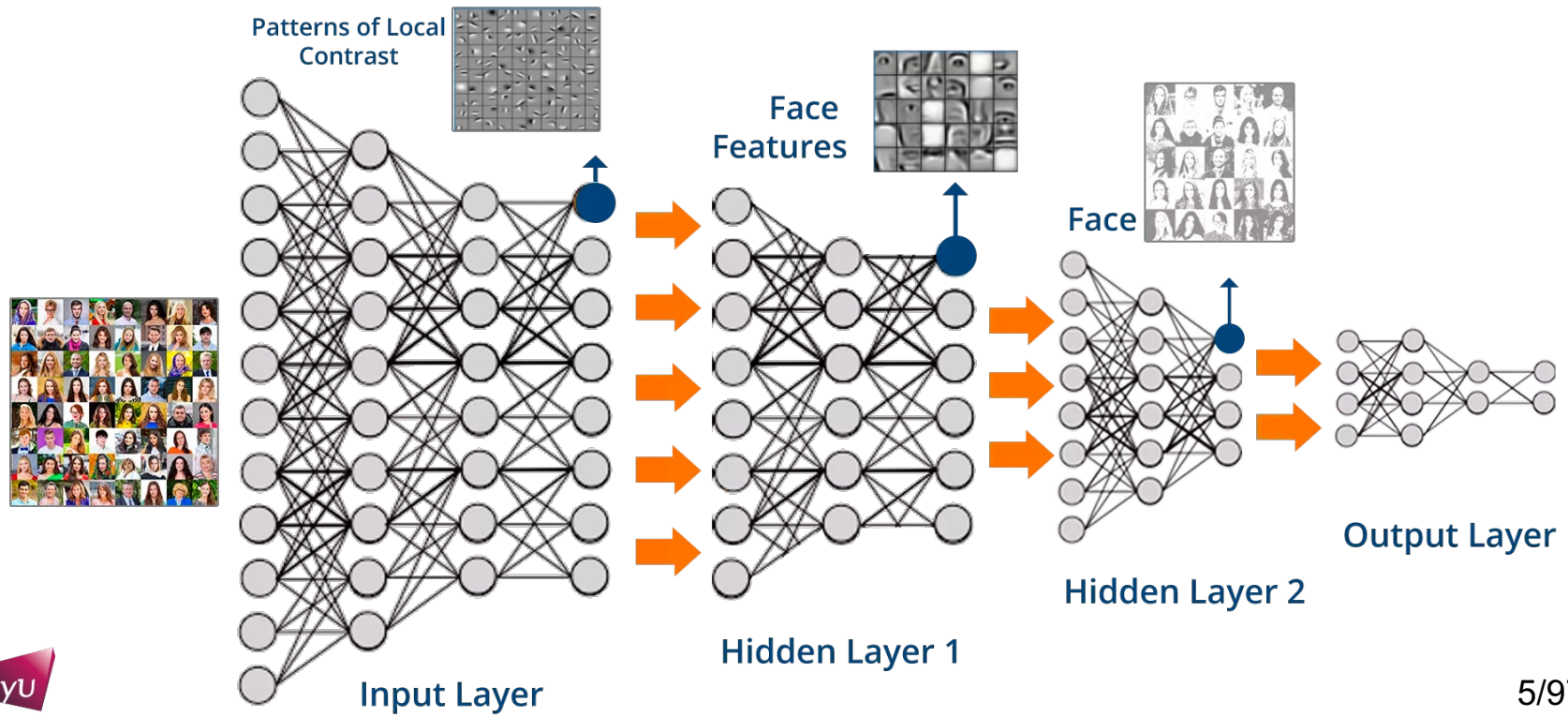
What is Deep Learning?



Deep learning?

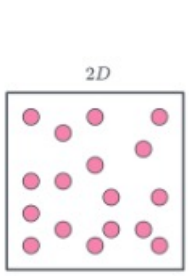
What is Deep Learning?

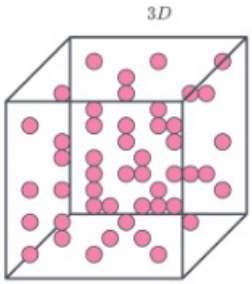
Deep learning use neural networks to simulate human like decision making

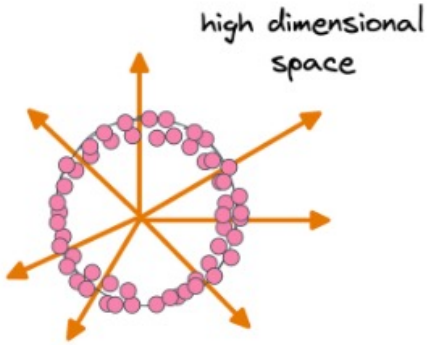


Why Deep Learning?

Limitations of fundamental machine learning algorithms



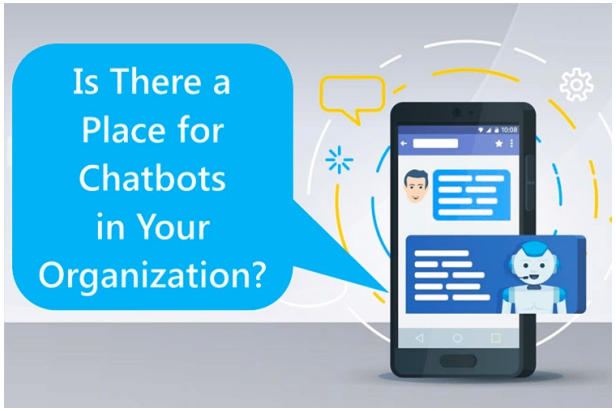




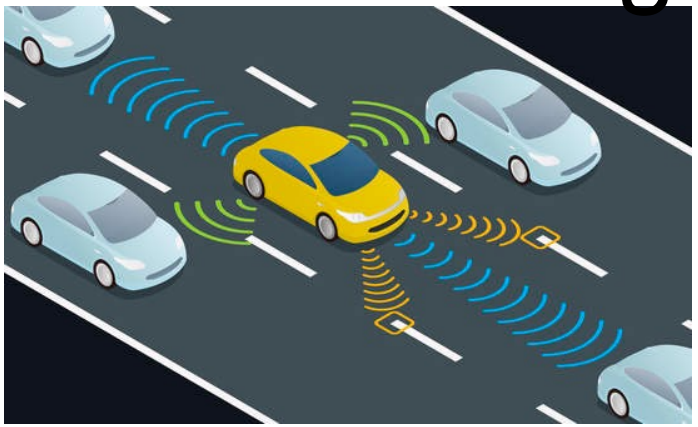
- Curse of Dimension!
- Large dataset
- Challenges of feature extraction
- Not able to solve complex problems such as image recognition, speech recognition, NLP,etc

Why Deep Learning?

Chatbot



Self-driving



Medicare



Image recognition



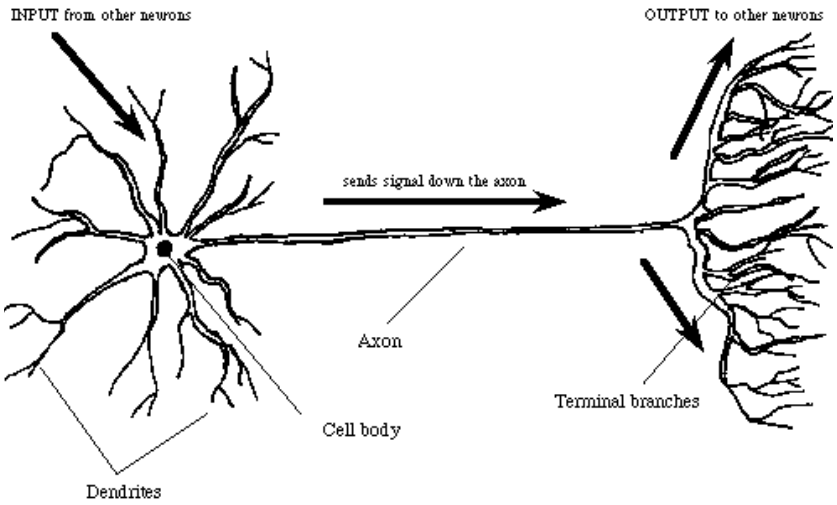
CityU
香港城市大學
City University of Hong Kong
專業 創新 與協全球
Professional · Creative
For The World

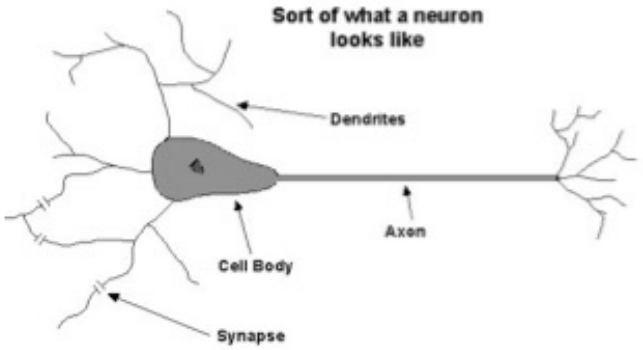
Why Deep Learning and NN?

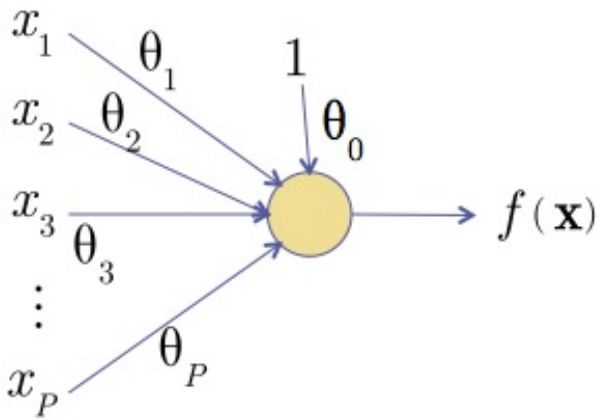
Huge market demand



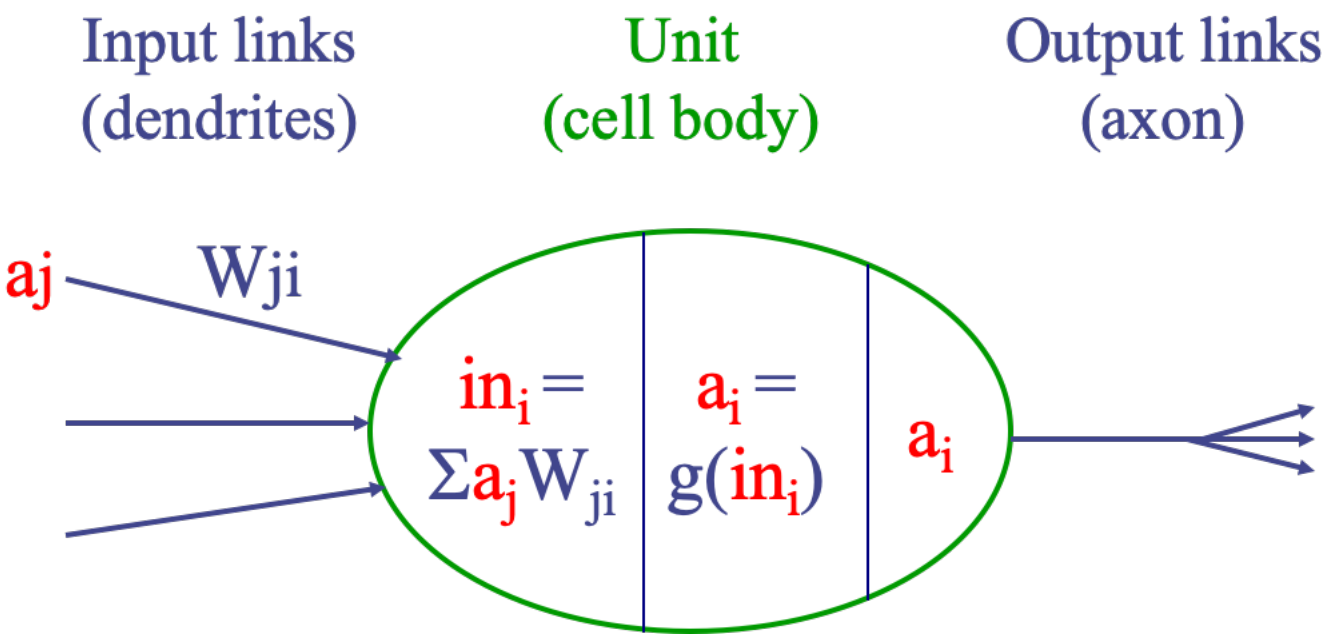
Biological Background



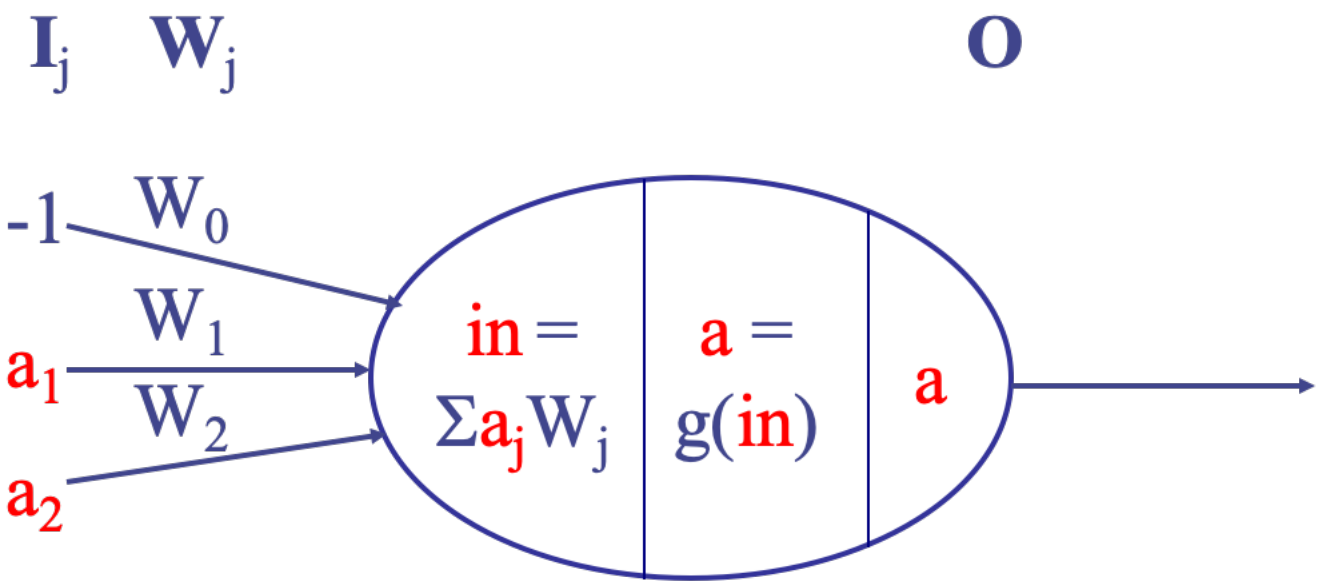




Artificial Neuron



Class I: Perceptron



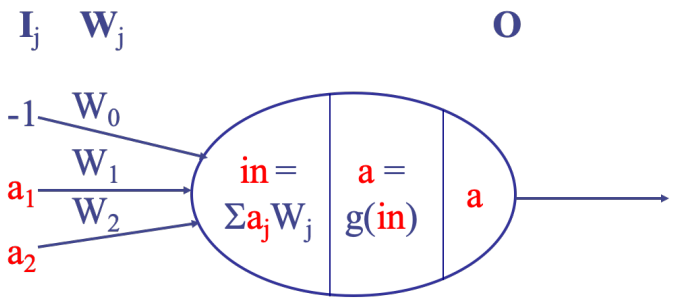
$$a = g(-W_0 + W_1 a_1 + W_2 a_2)$$

$$g(in) = \begin{cases} 0, & in<0 \\ 1, & in>0 \end{cases}$$

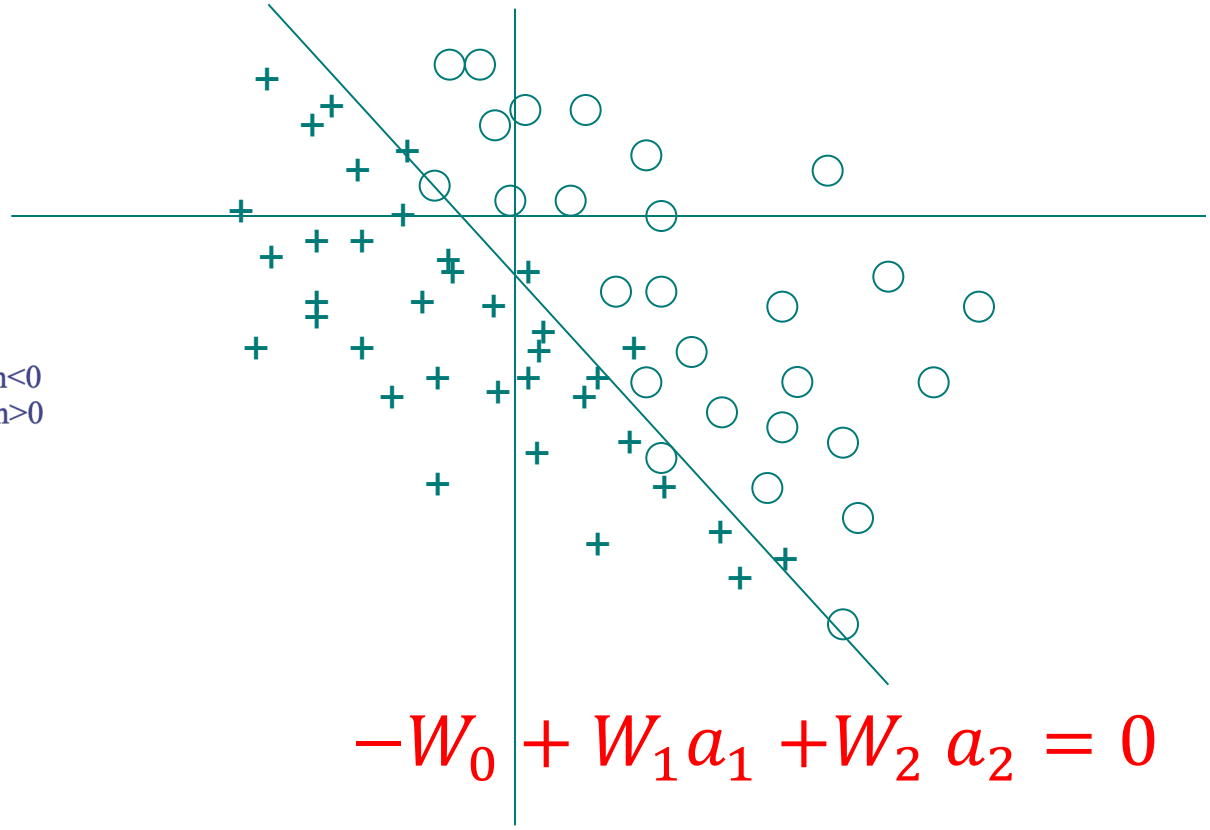
Step function

5

Use a trained model



$a = g(-W_0 + W_1 a_1 + W_2 a_2)$ $\quad g(in) = \begin{cases} 0, & in<0 \\ 1, & in>0 \end{cases}$

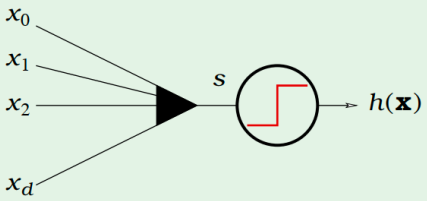
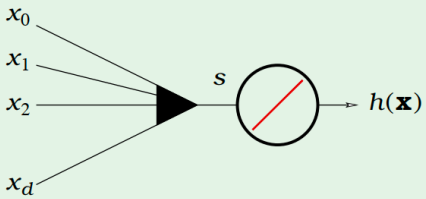
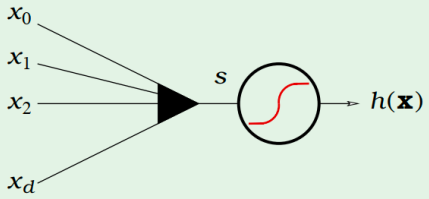


$$-W_0 + W_1 a_1 + W_2 a_2 = 0$$

Perceptron's Step Function

How to deal with this 's'?

$$s = \sum_{i=0}^{d} w_i x_i$$

Perceptron	linear regression	logistic regression
linear classification		
$h(\mathbf{x}) = \text{sign}(s)$	$h(\mathbf{x}) = s$	$h(\mathbf{x}) = g(s)$
		
Step function		Sigmoid function
Output Range $\{+1,-1\}$ or $\{0,-1\}$	$(+\infty,-\infty)$	$g(s) = 1/(1+\exp(-s))$ $(0,1)$
Hard classification	Continuous prediction	Probability-based classification

Perceptron's Capability

Logical Function

AND function:

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

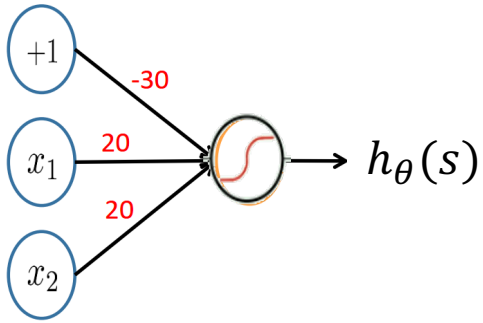
$$y = g(\theta^T x)$$
$$= g(-30 + 20x_1 + 20x_2)$$
$$= \frac{1}{1 + e^{-30+20x_1+20x_2}}$$

How perceptron can achieve AND?

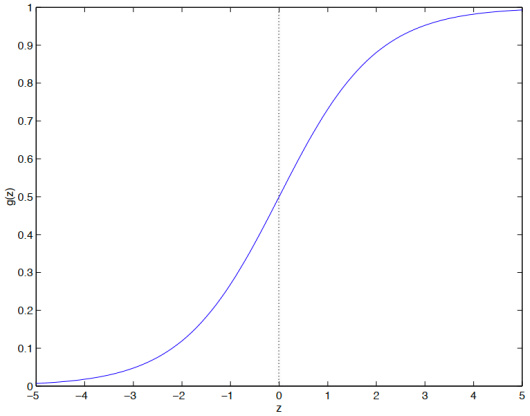
Simple example: AND

$x_1, x_2 \in \{0, 1\}$

$y = x_1 \text{ AND } x_2$



$h_\theta(s) = g(-30 + 20x_1 + 20x_2)$

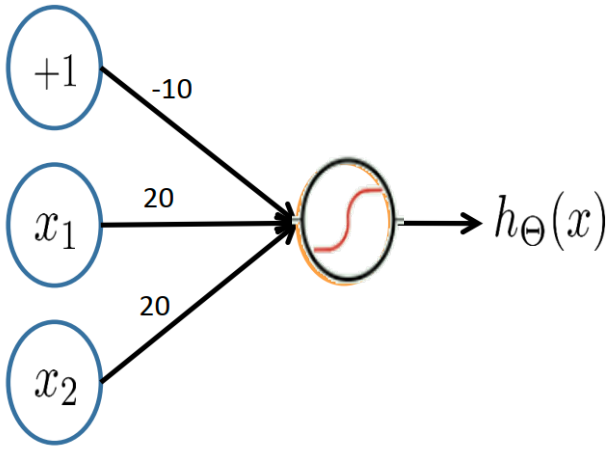


x_1	x_2	$h_\theta(s)$
0	0	g(-30) ~0
0	1	g(-10) ~0
1	0	g(-10) ~0
1	1	g(10) ~1

$$h_\theta(s) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Perceptron's Capability

Example: OR function



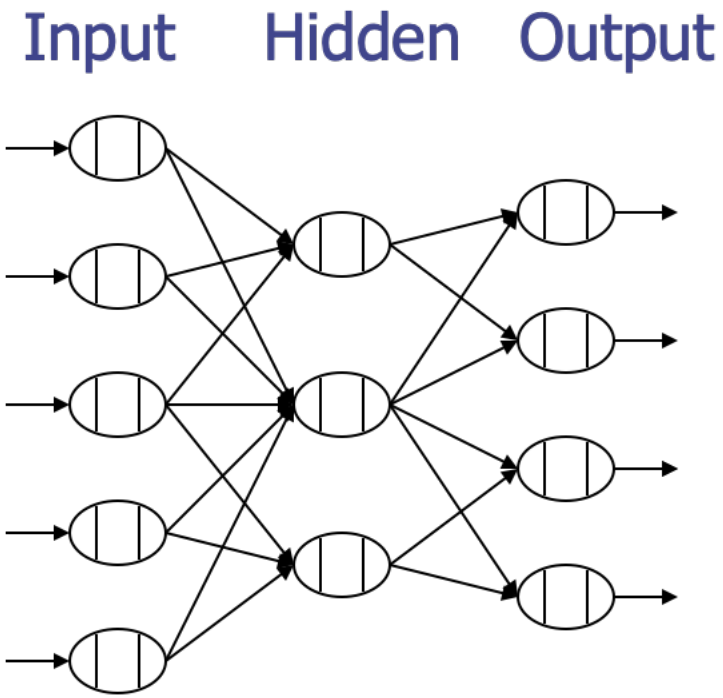
g(-10 + 20x_1 + 20x_2)

x_1	x_2	$h_\Theta(x)$
0	0	g(-10) ~0
0	1	g(10) ~1
1	0	g(10) ~1
1	1	g(30) ~1

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

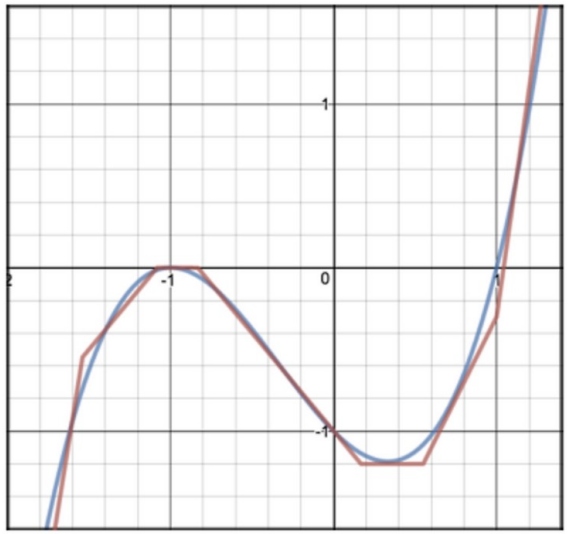
Class II: Multi-layer Feed-forward Networks

- Multiple layers:
  - hidden layer(s)
- Feed-forward:
  - Output links only connected to input links in the next layer
- Complex non-linear functions can be represented



Class II Fundation: Universal Approximation Theorem

- A feedforward neural network with a single hidden layer and enough neurons can approximate any continuous function on a compact subset of $\mathbb{R}^n$, given a suitable nonlinear activation function.



$$n_1(x) = Relu(-5x - 7.7)$$
$$n_2(x) = Relu(-1.2x - 1.3)$$
$$n_3(x) = Relu(1.2x + 1)$$
$$n_4(x) = Relu(1.2x - .2)$$
$$n_5(x) = Relu(2x - 1.1)$$
$$n_6(x) = Relu(5x - 5)$$

$$Z(x) = -n_1(x) - n_2(x) - n_3(x) + n_4(x) + n_5(x) + n_6(x)$$

A complex case

XOR function:

y=1 only when two x are same

x1	x2	y
0	0	1
0	1	0
1	0	0
1	1	1

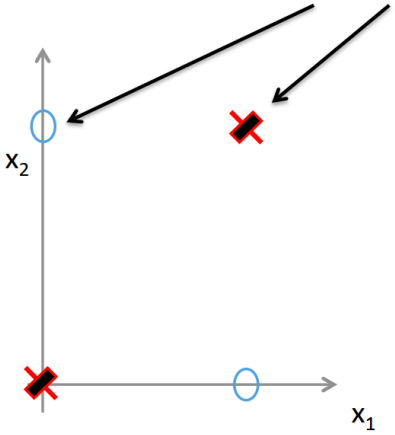
XNOR function:

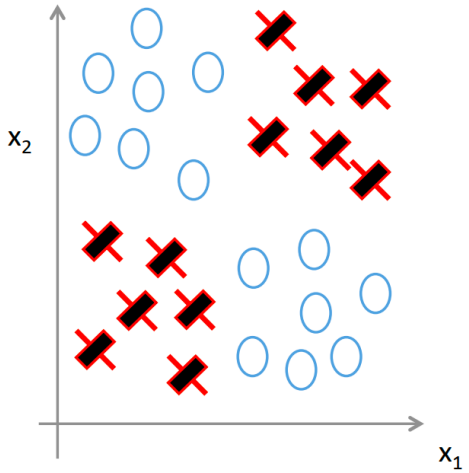
y=1 only when two x are different

Why single layer fails?

Non-linear classification example: XOR/XNOR

x_1, x_2 are binary (0 or 1).





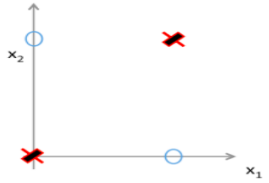
$$y = x_1 \text{ XOR } x_2$$

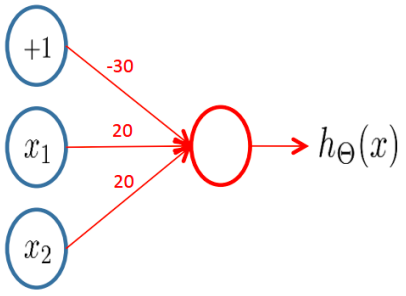
$$x_1 \text{ XNOR } x_2$$

$$\text{NOT } (x_1 \text{ XOR } x_2)$$

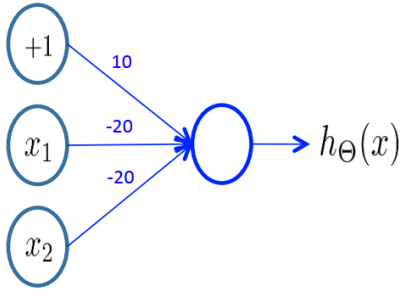
A complex case

Putting it together: x_1 XOR x_2

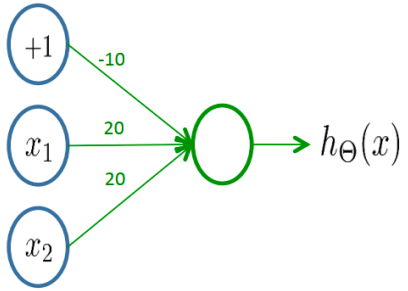




x_1 AND x_2



(NOT x_1) AND (NOT x_2)

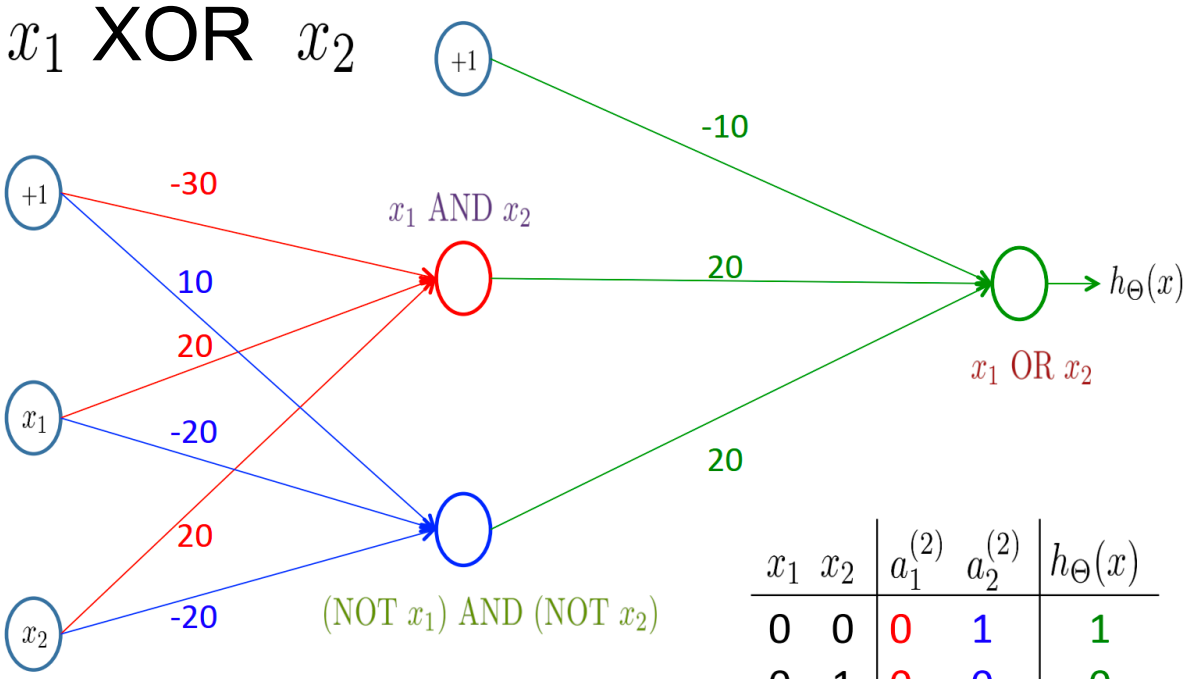


x_1 OR x_2



x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\Theta(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

A complex case



x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\Theta(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

A more complex case – multi output

Multiple output units: One-vs-all.



Pedestrian



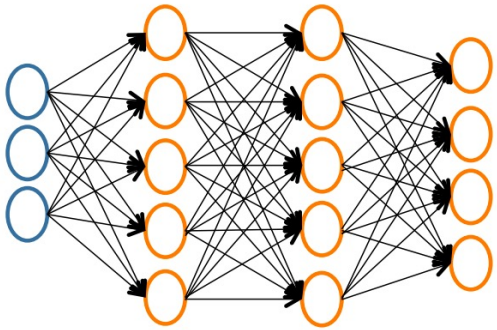
Car



Motorcycle



Truck



$$h_\Theta(x) \in \mathbb{R}^4$$

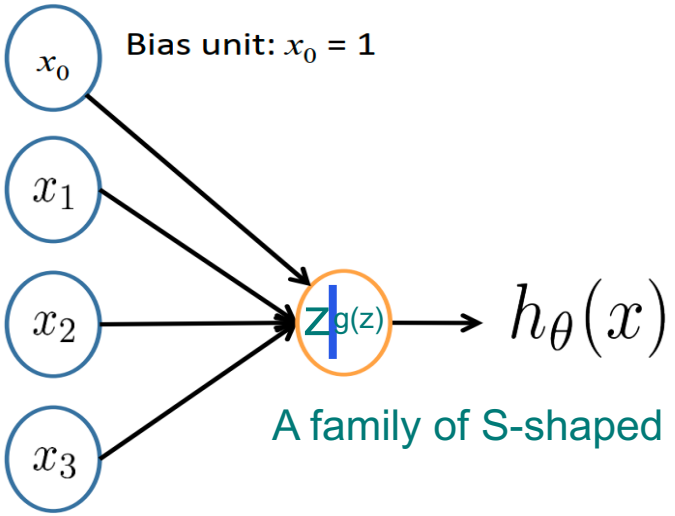
Want $h_\Theta(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.

when pedestrian when car when motorcycle

Build an Artificial Neural Network

- Design a structure
  - Number of input (input nodes)
  - Number of hidden layer
  - Number of nodes per hidden layer
  - Number of output nodes (targets)
- Decide the node activation function
  - From node input to node output
- Decide the parameters
  - Coefficients from previous layer output to next layer input
  - Parameters of activation function (if any)

ANN Model Representation



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

Activation function

A family of S-shaped functions
- Sigmoid function
- Logistic function

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z)).$$

Sigmoid (logistic) activation function.

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Model Representation

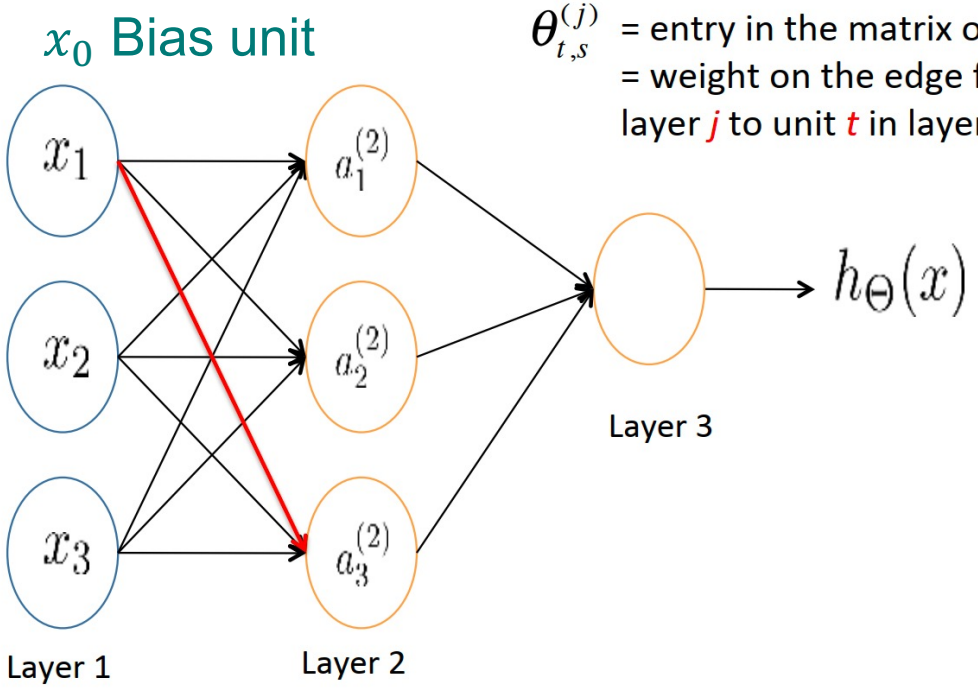
Neural Network

$a_i^{(j)}$ = "activation" of unit i in layer j

$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer j+1

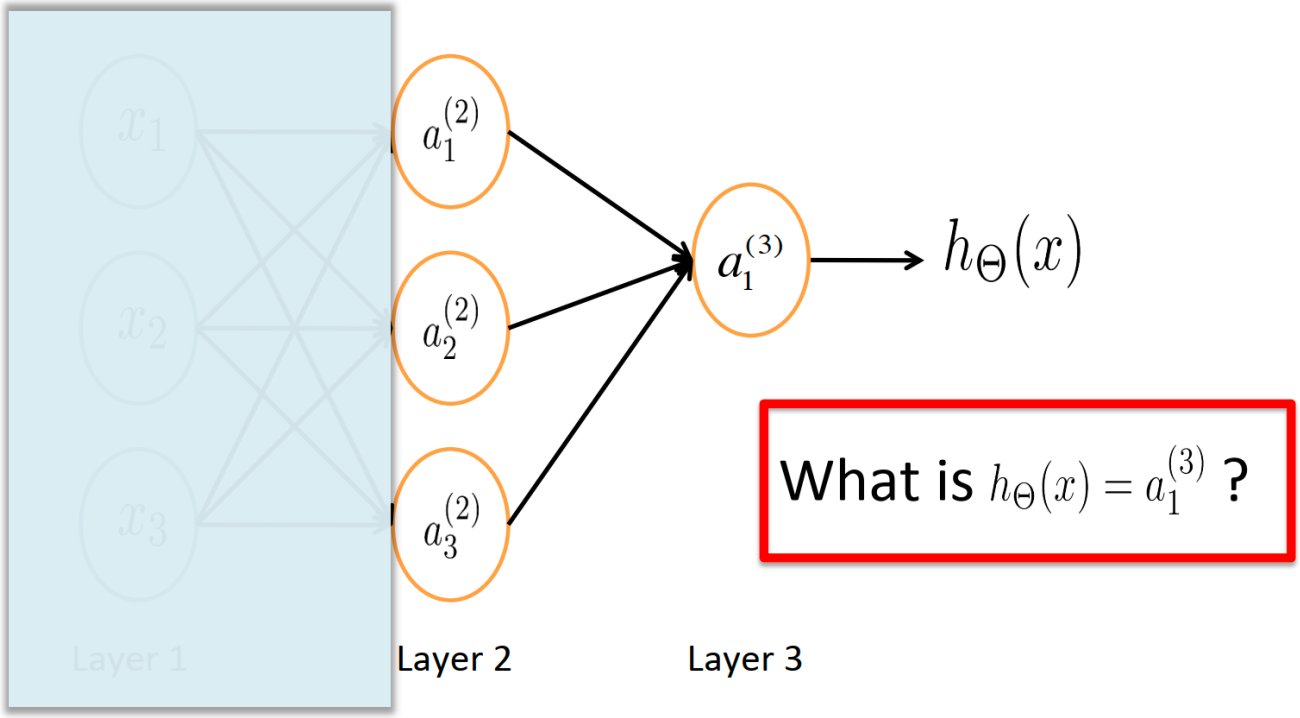
$\theta_{t,s}^{(j)}$ = entry in the matrix of weights
= weight on the edge from unit *s* in layer *j* to unit *t* in layer *j*+1

Where does $\theta_{3,1}^{(1)}$ go?



Model Representation

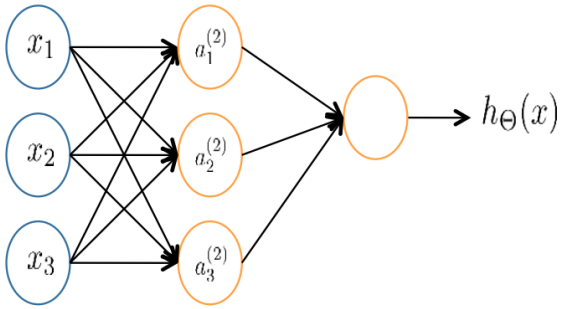
Neural network learns its own features



$$h_\Theta(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

Model Representation

Neural Network



$a_i^{(j)}$ = "activation" of unit i in layer j

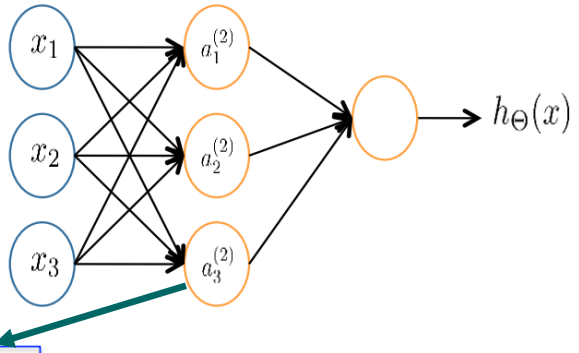
$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer j+1

$$a_1^{(2)} = g(\Theta_{10}^{(1)}x_0 + \Theta_{11}^{(1)}x_1 + \Theta_{12}^{(1)}x_2 + \Theta_{13}^{(1)}x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)}x_0 + \Theta_{21}^{(1)}x_1 + \Theta_{22}^{(1)}x_2 + \Theta_{23}^{(1)}x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)}x_0 + \Theta_{31}^{(1)}x_1 + \Theta_{32}^{(1)}x_2 + \Theta_{33}^{(1)}x_3)$$
$$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)}a_0^{(2)} + \Theta_{11}^{(2)}a_1^{(2)} + \Theta_{12}^{(2)}a_2^{(2)} + \Theta_{13}^{(2)}a_3^{(2)})$$

If network has s_j units in layer j, and s_{j+1} units in layer j+1, then $\Theta^{(j)}$ will be of dimension $s_{j+1} \times (s_j + 1)$

Model Representation

Forward propagation: Vectorized implementation



$$
\begin{aligned}
a_1^{(2)} &= g\left(\Theta_{10}^{(1)}x_0 + \Theta_{11}^{(1)}x_1 + \Theta_{12}^{(1)}x_2 + \Theta_{13}^{(1)}x_3\right) \\
a_2^{(2)} &= g\left(\Theta_{20}^{(1)}x_0 + \Theta_{21}^{(1)}x_1 + \Theta_{22}^{(1)}x_2 + \Theta_{23}^{(1)}x_3\right) \\
a_3^{(2)} &= g\left(\Theta_{30}^{(1)}x_0 + \Theta_{31}^{(1)}x_1 + \Theta_{32}^{(1)}x_2 + \Theta_{33}^{(1)}x_3\right)
\end{aligned}
$$

$$
x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}
$$

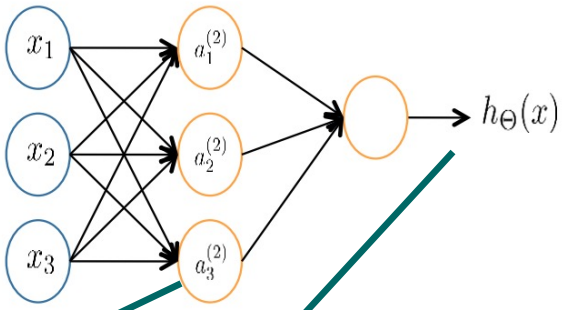
$$
z^{(2)} = \Theta^{(1)}x
$$

Matrix operation

$$
a^{(2)} = g\left(z^{(2)}\right)
$$

Model Representation

Forward propagation: Vectorized implementation



$$a^{(1)} = x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix}$$

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_\Theta(x) = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

$$z^{(2)} = \Theta^{(1)} x$$
$$a^{(2)} = g(z^{(2)})$$

$a_i^{(j)}$ = "activation" of unit i in layer j

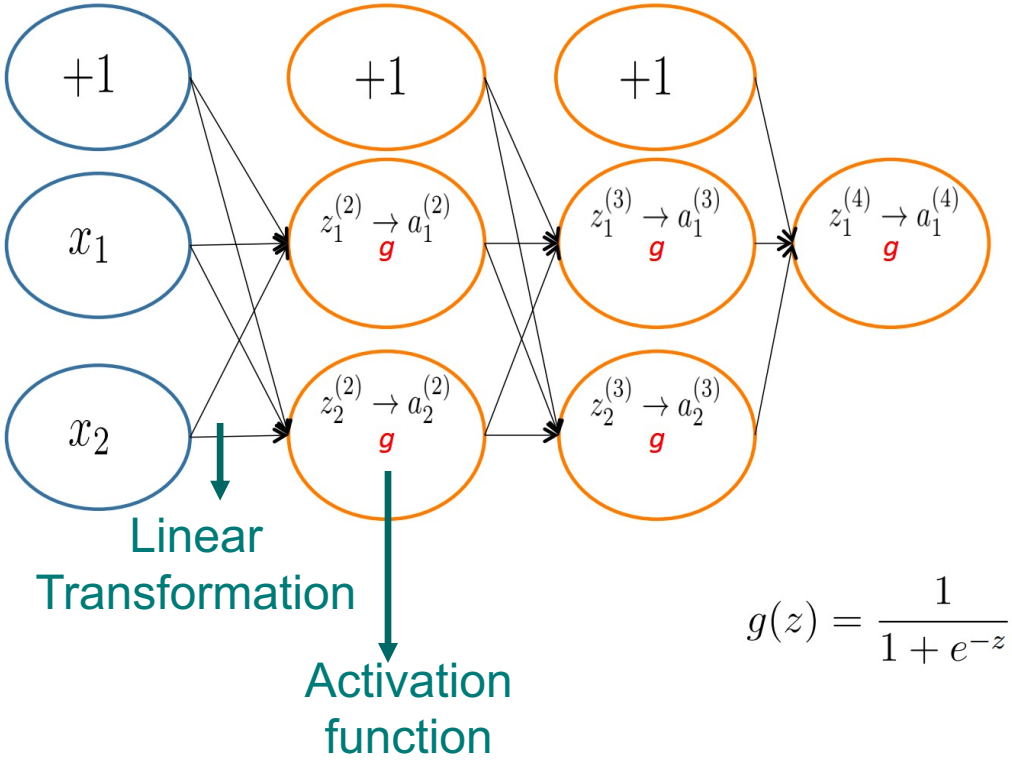
$\Theta^{(j)}$ = matrix of weights controlling function mapping from layer j to layer $j+1$

Add $a_0^{(2)} = 1.$

$$z^{(3)} = \Theta^{(2)} a^{(2)}$$
$$h_\Theta(x) = a^{(3)} = g(z^{(3)})$$

NN with 2 hidden layers

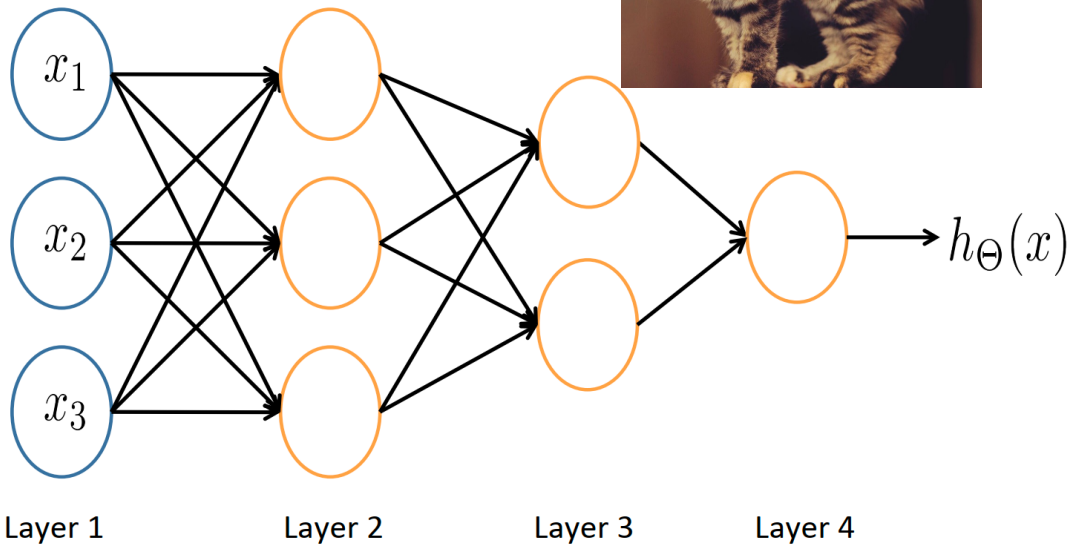
Forward Propagation

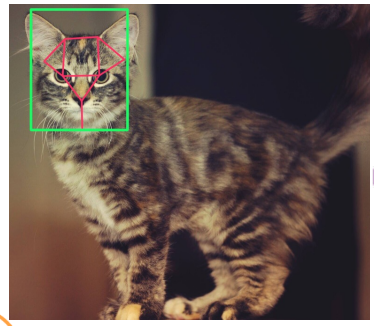


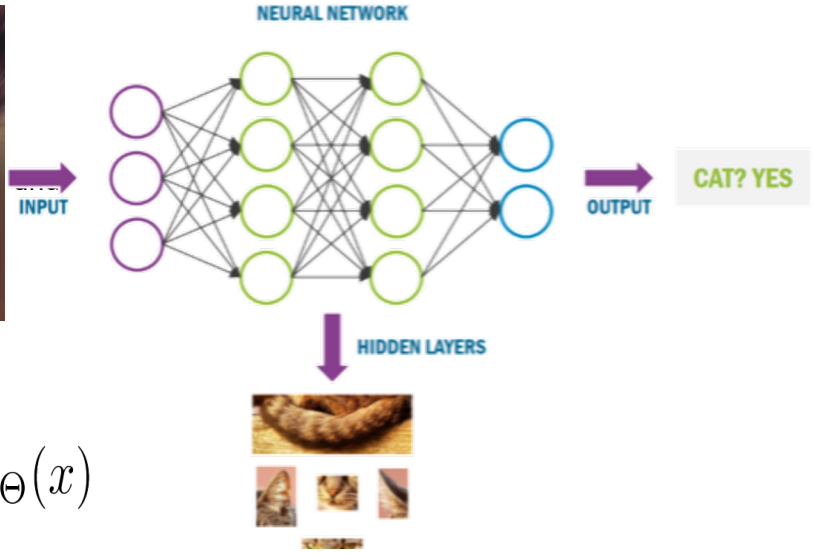
$$g(z) = \frac{1}{1 + e^{-z}}$$

NN learn complex features

Network architectures







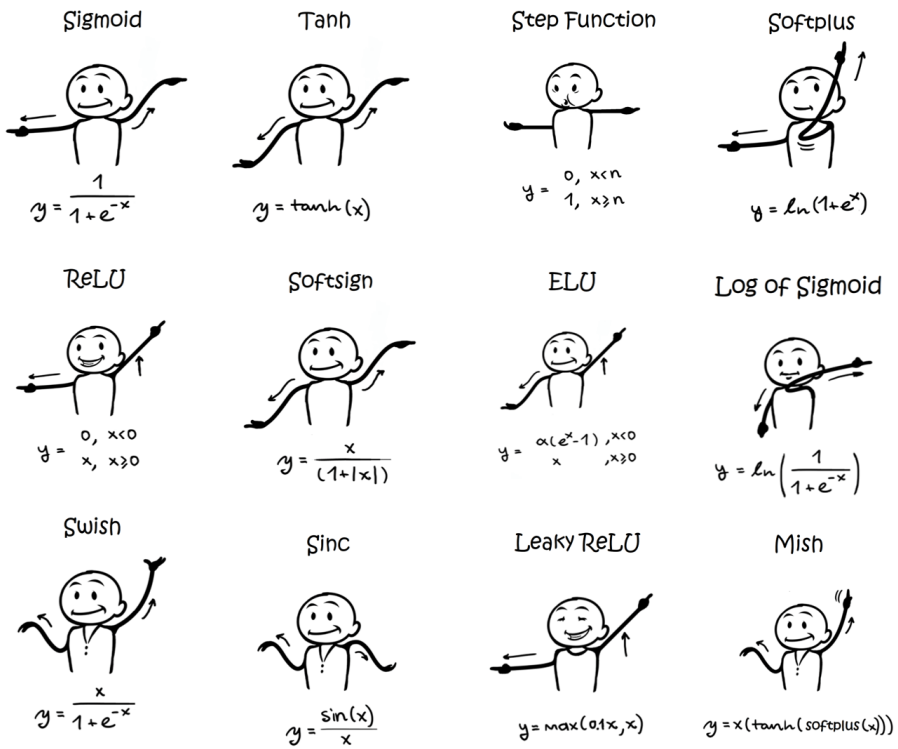
1. Edges
2. Patches
3. Surfaces, and
4. Objects

Neural networks learn complex features through their hidden layers

Activation Function

Activation function: Magical "g"

The activation function is a mathematical **"gate"** in between the input feeding the current neuron and its output going to the next layer



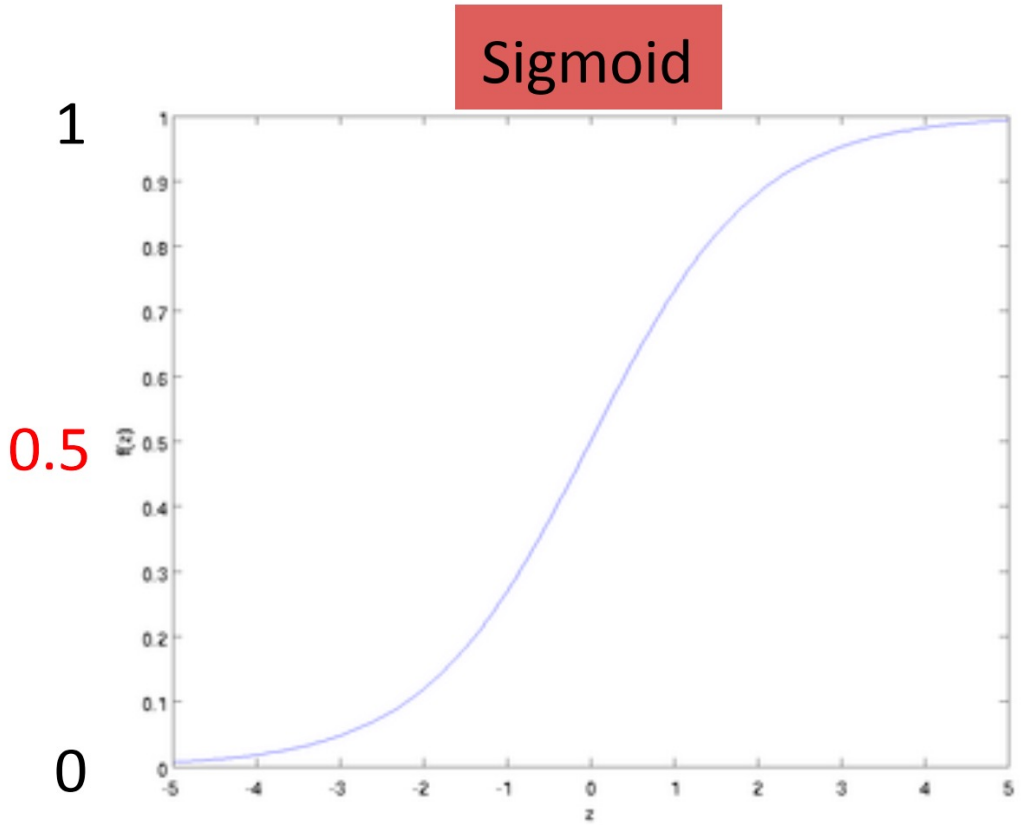
Sigmoid Activation Function

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z)).$$

- Maps any real-valued input → output between 0 and 1
- "S"-shaped curve

- Binary classification (prob)



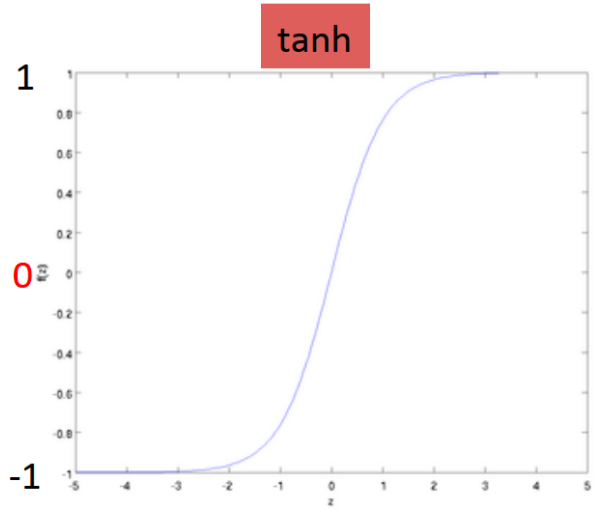
Tangent activation function

Another commonly used activation function is the hyperbolic tangent, tanh

- The tanh(z) function is a rescaled version of the sigmoid, and its output range is [− 1,1] instead of [0,1].

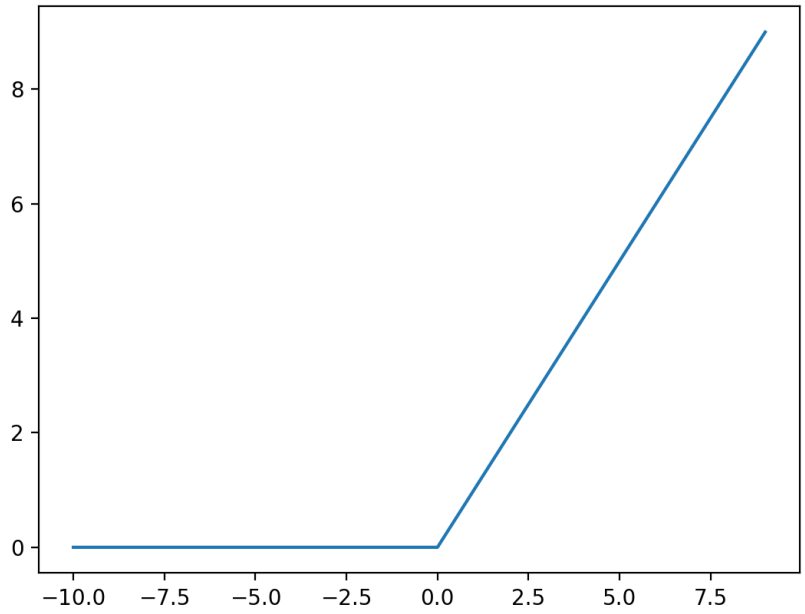
$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}},$$

- $f'(z) = 1 - (f(z))^2$



ReLU activation function

Rectifier Linear Unit

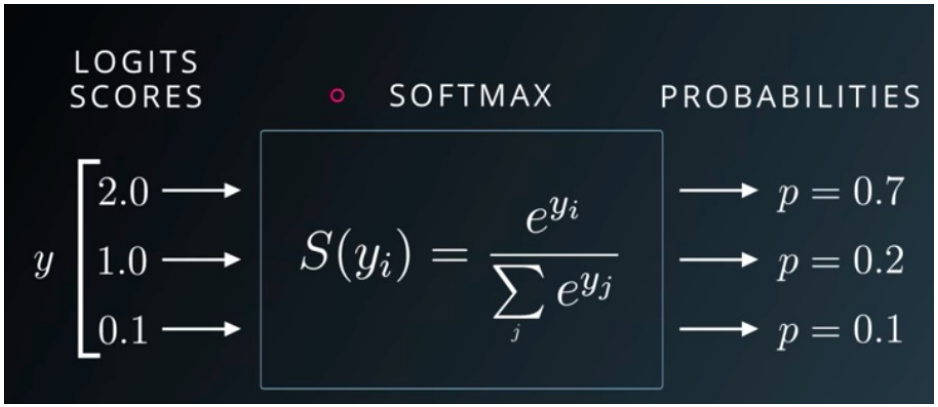


ReLU

$$f(z) = \max(0, z)$$

- Sparse Representation
- Computational Efficiency

Softmax Activation



Multiple output units: One-vs-all.

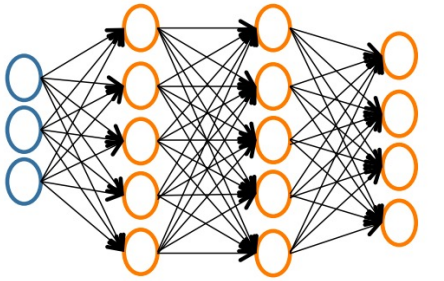


Pedestrian Car Motorcycle

Usually used for last layer with its output representing the possibility of classes

Model Training

Neural Network (Classification)



$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

L = total no. of layers in network

s_l = # of units (not counting bias unit) in layer l

Binary classification

$y = 0$ or 1

1 output unit

Multi-class classification (K classes)

$y \in \mathbb{R}^K$ E.g. $\begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$

pedestrian car motorcycle truck

K output units

Cost Function (Objective)

Cost function

L = total no. of layers in network

s_l = no. of units (not counting bias unit) in layer l

One output

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2$$

Training Error

Regularization

Prevent overfitting

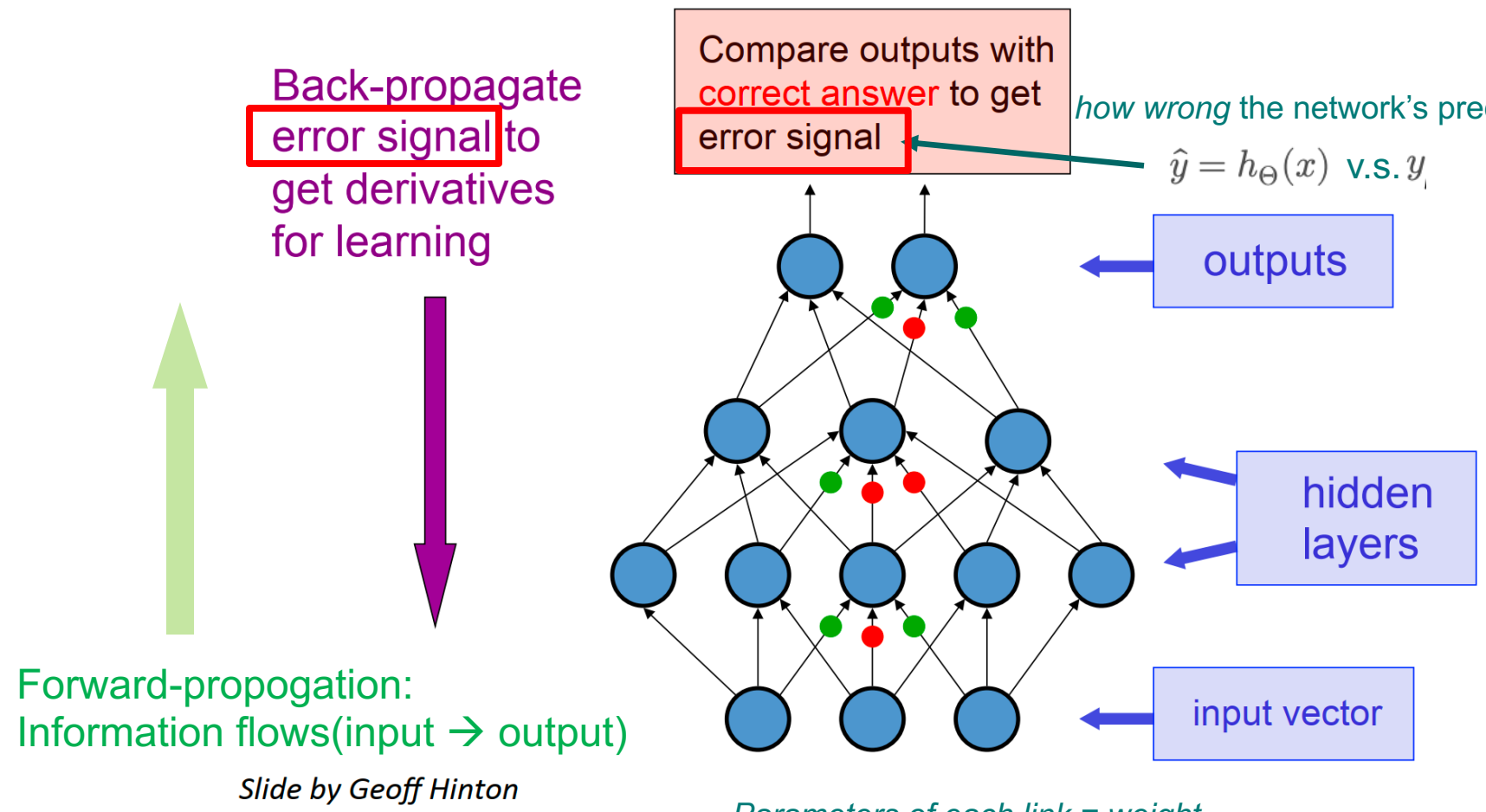
K outputs

$$h_\Theta(x) \in \mathbb{R}^K \quad (h_\Theta(x))_i = i^{th} \text{ output}$$

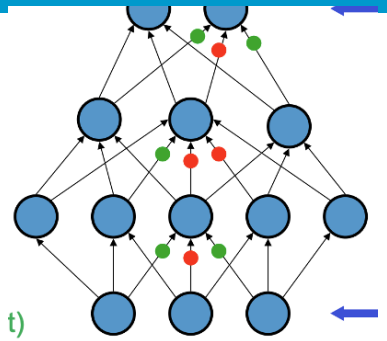
$$J(\Theta) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log(h_\Theta(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_\Theta(x^{(i)}))_k)\right]$$

$$+\frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}(\Theta_{ji}^{(l)})^2$$

Back propagation



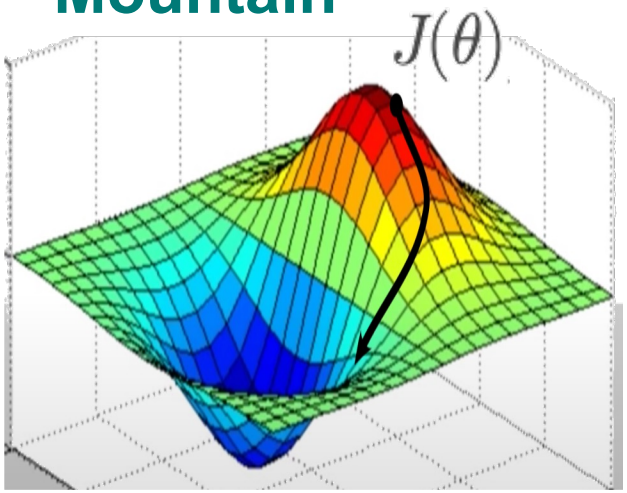
Gradient Descent:
What if p is large? (e.g., n/2)

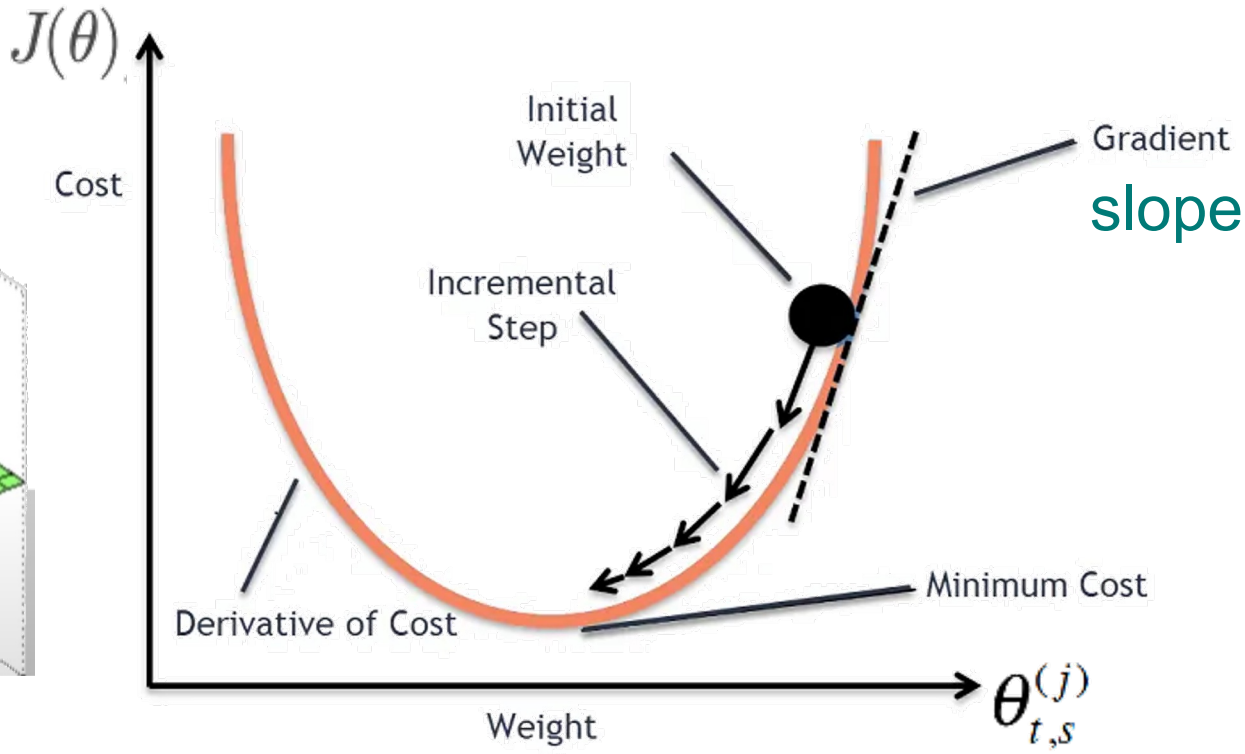


$J(\theta)$ $\theta_{t,s}^{(j)}$

The **derivatives** of **Cost function** to **weight**

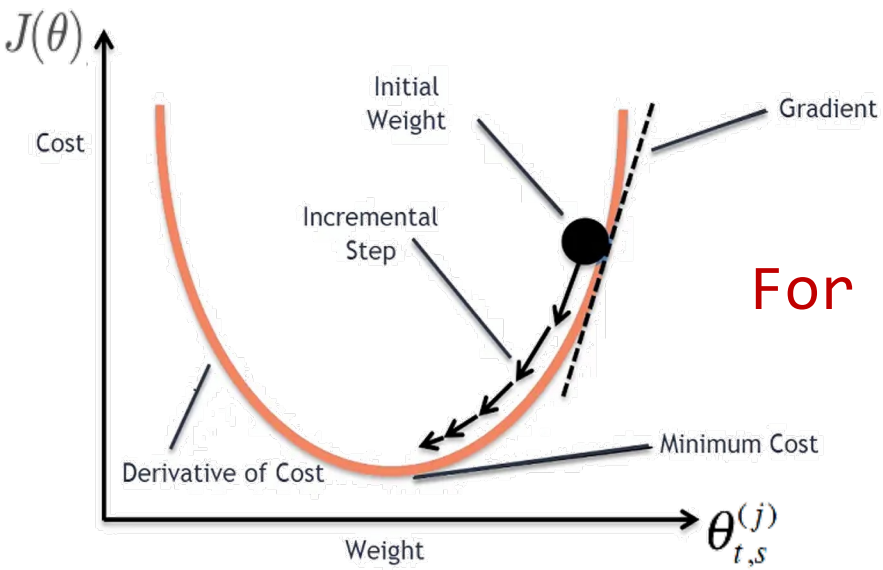
Mountain





Gradient Descent:

- Solution: Iterative Methods
  - Gradient Descent:



Slope = 0

Learning rate

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{\partial J}{\partial \theta}$$

For t from 0 until *convergence*

Partial Derivative

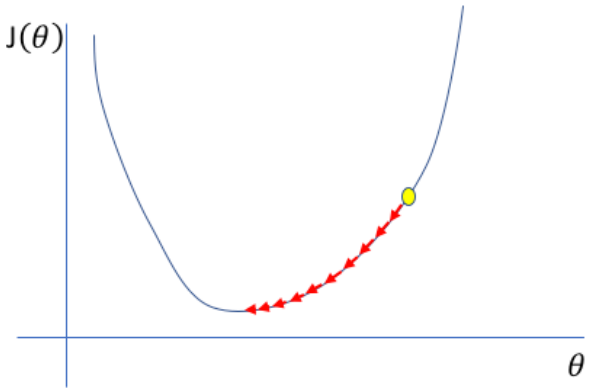
Gradient Descent: Learning Rate

- Gradient Descent
  - Learning rate

Learning rate

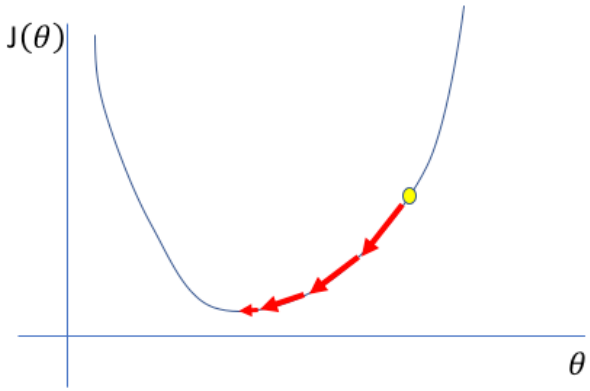
$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{\partial J}{\partial \theta}$$

Too low



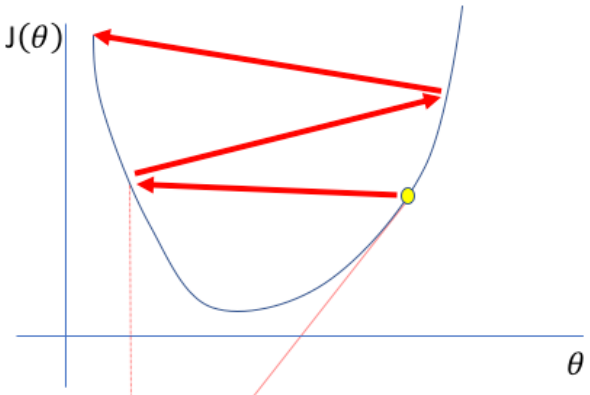
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high



Too large of a learning rate causes drastic updates which lead to divergent behaviors

Gradient computation given cost function

Gradient computation

$$J(\Theta) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log h_\theta(x^{(i)})_k + (1 - y_k^{(i)}) \log(1 - h_\theta(x^{(i)})_k)\right]$$
$$+\frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}(\Theta_j^{(l)})^2$$

$$\min_{\Theta} J(\Theta)$$

Need code to compute:

$J(\Theta)$	$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$

$\nabla_\Theta J(\Theta)$

$$\Theta := \Theta - \alpha \nabla_\Theta J(\Theta)$$

$$\theta := \theta - \alpha \frac{d}{d\theta} J(\theta).$$

Overfitting of NN

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2$$

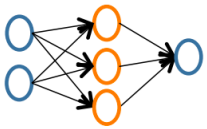
Training Error

Regularization

Prevent overfitting

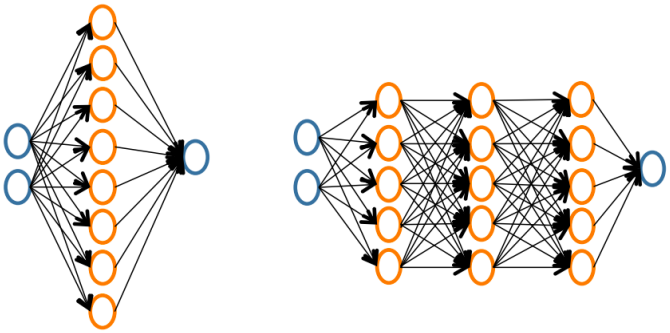
Neural networks and overfitting

"Small" neural network (fewer parameters; more prone to underfitting)



Computationally cheaper

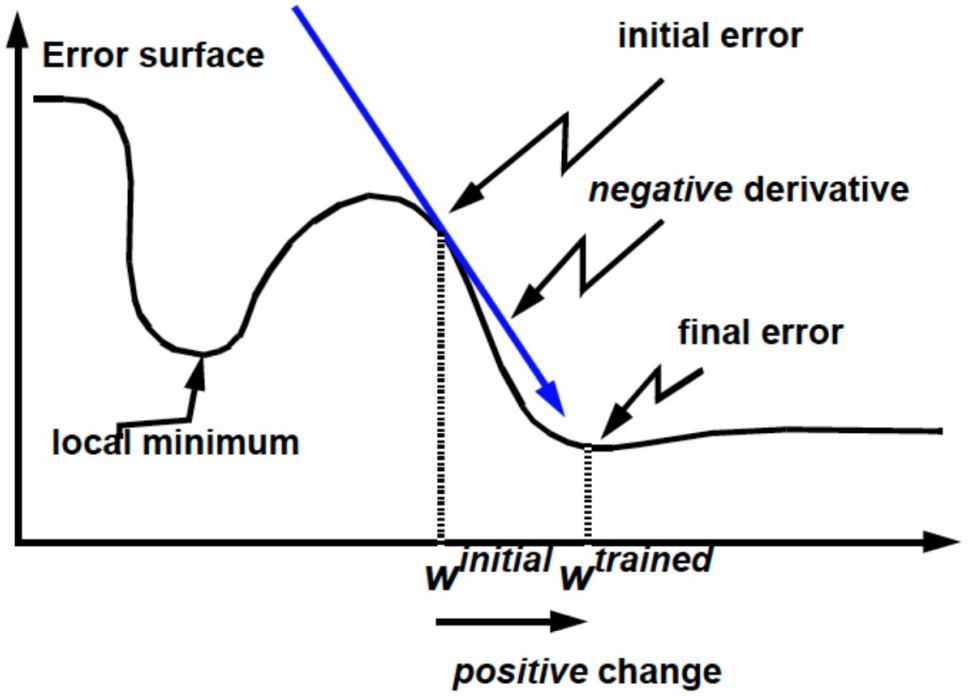
"Large" neural network (more parameters; more prone to overfitting)

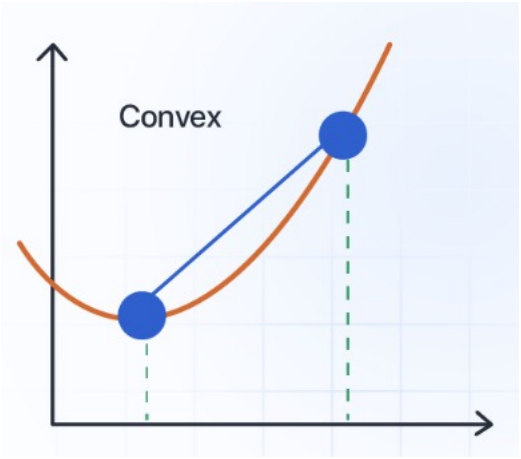


Computationally more expensive.

Use regularization (λ) to address overfitting.

Local Optimal Parameters





Random initialization

Example

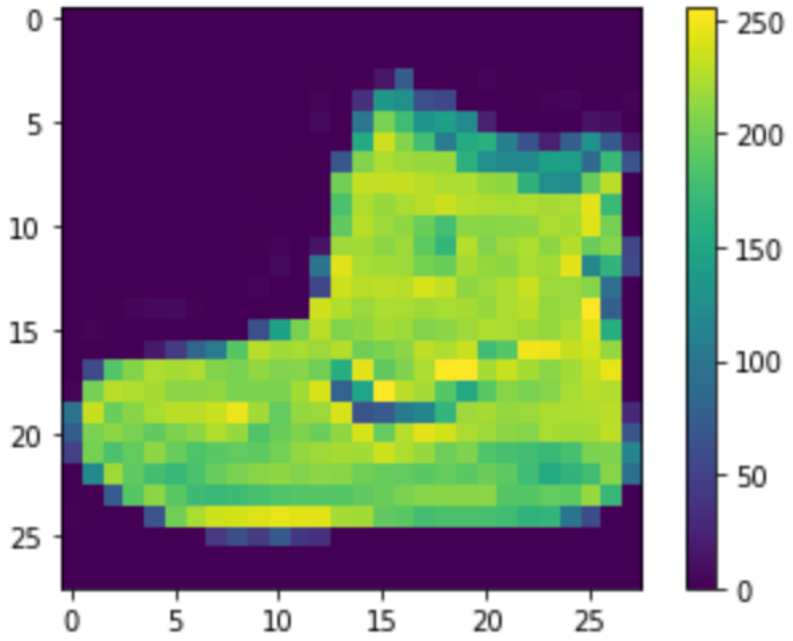
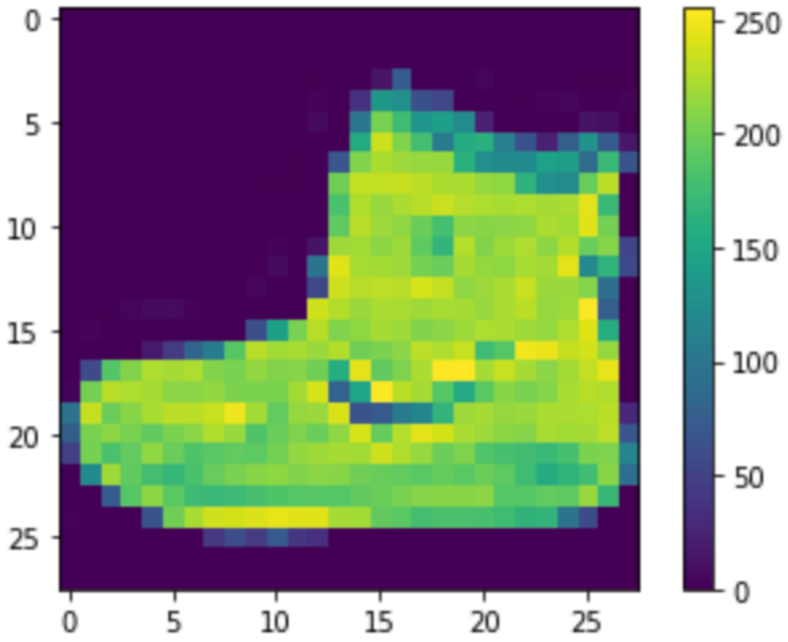


Image classification

Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Example



Object has 28*28 features (pixels)

Example

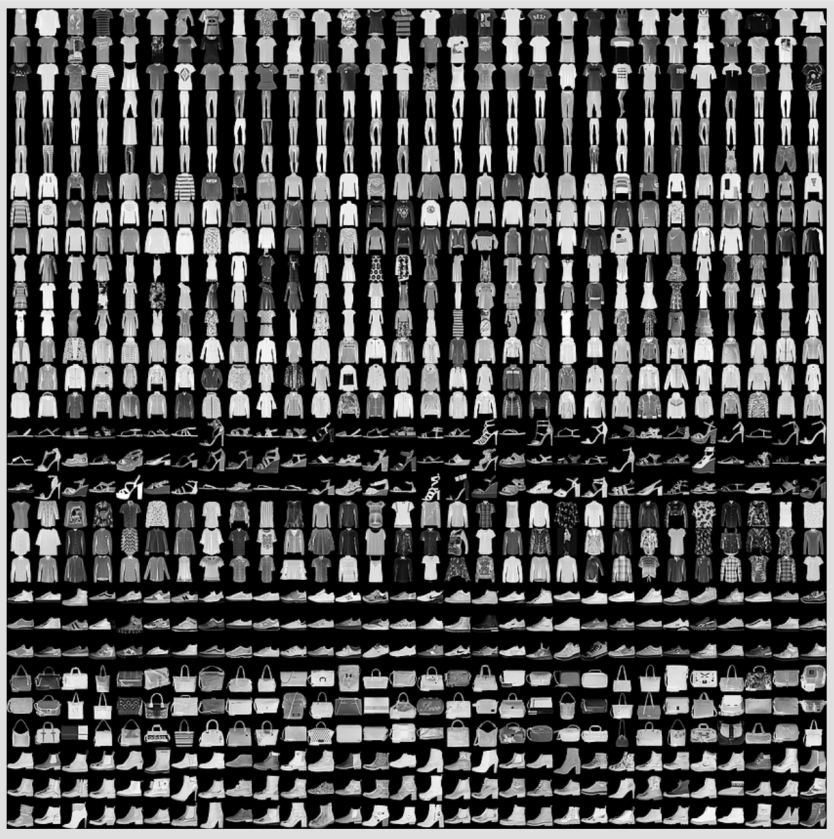
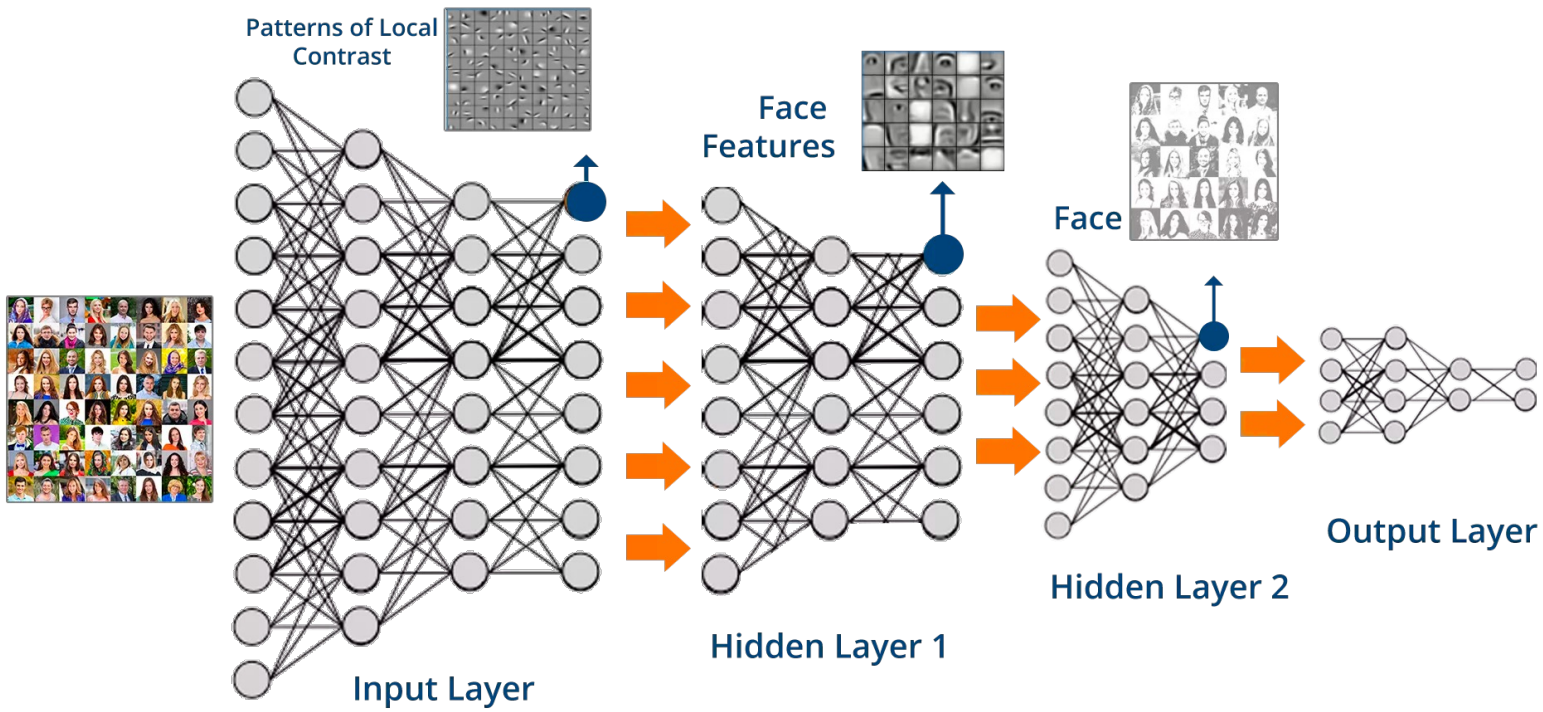


Figure 1. Fashion-MNIST samples (by Zalando, MIT License).

60,000 objects with labels for model training

Example

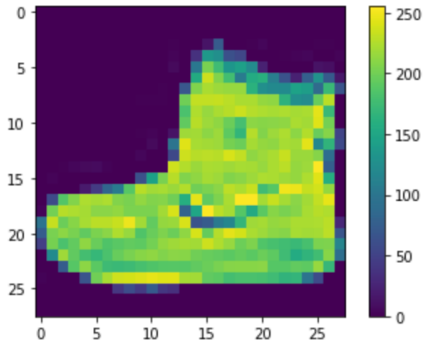


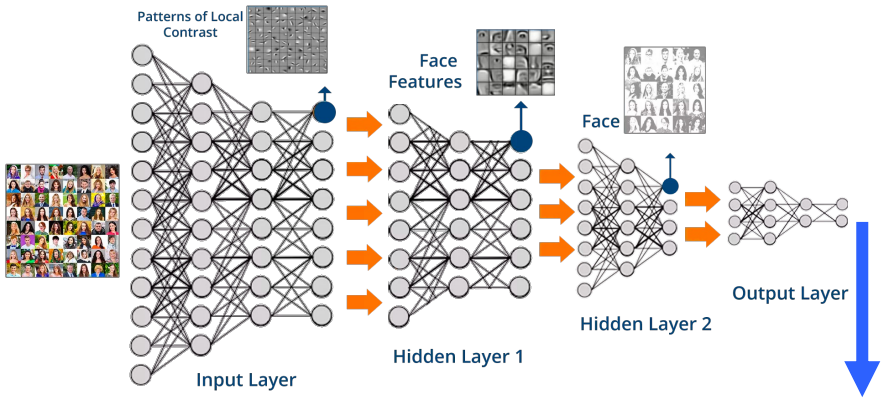
Identify the following parameters:

- Input layer dimension (for features input)
- Number of hidden layers
- Number of nodes in hidden layers
- Activation function
- Loss function (objective function)

Example

Testing object → Trained ANN model





Label	Class
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Output y

$$y = (p_0, p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8, p_9)$$

$$p_i : probability\ of\ class\ i$$

If the max probability is p_j,
prediction result is class j

IS6400 Business Data Analytics

ZHANG Yanyu

City University of Hong Kong

Lecture 11

- ❑ Intro-Deep Learning
  - ❑ ANN
  - ❑ RNN, LSTM
  - ❑ NLP

Tutorial 11:

- ❑ Deep Learning

FFNN

Static Input and Static Output

Multiple output units: One-vs-all.



Pedestrian



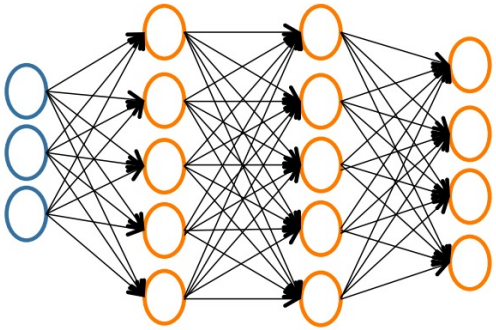
Car



Motorcycle



Truck



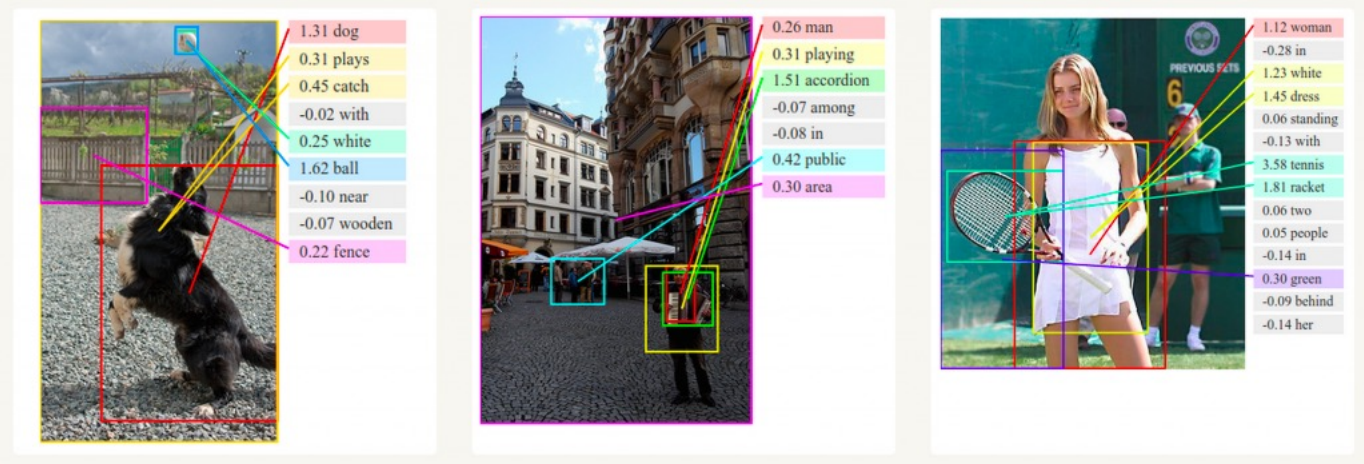
$h_\Theta(x) \in \mathbb{R}^4$

Want $h_\Theta(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, $h_\Theta(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$, etc.

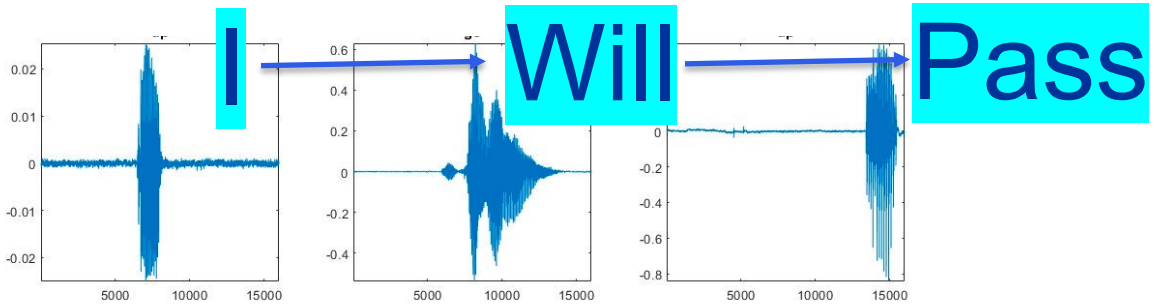
when pedestrian when car when motorcycle

NN for Sequence Analysis

What if I want to analyze something **in a sequence**?

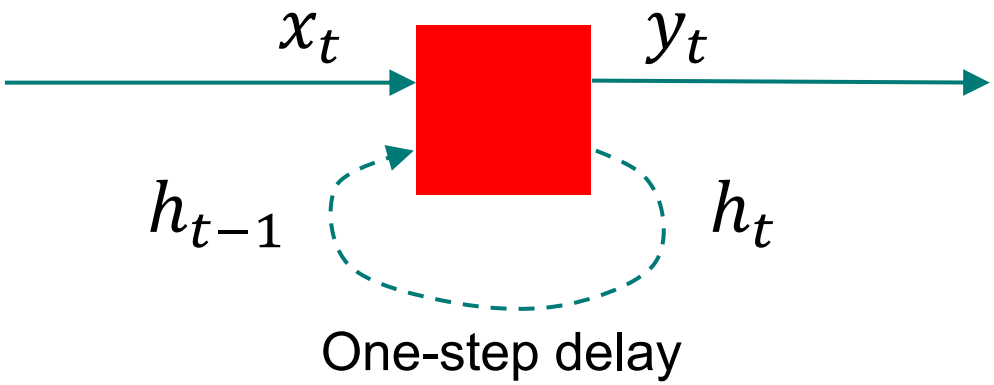


What if I want to analyze human speech?



Recurrent Neural Networks (RNNs)

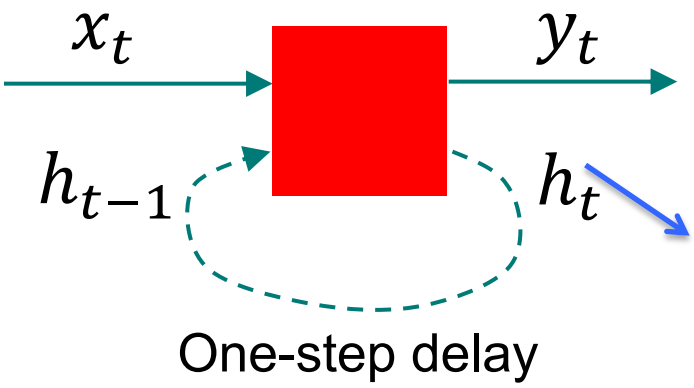
Recurrent networks introduce cycles and a notion of time.

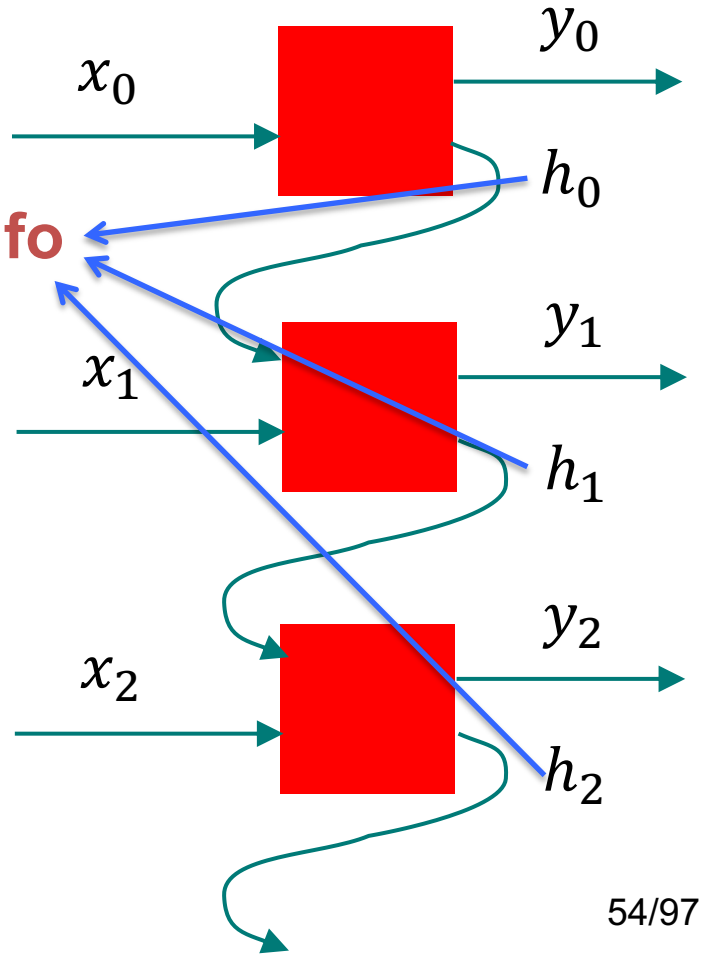


- They are designed to process sequences of data $x_1, \ldots, x_n$ and can produce sequences of outputs $y_1, \ldots, y_m$.

Unrolling RNNs

RNNs can be unrolled across multiple time steps.





This produces a **DAG** which supports backpropagation.

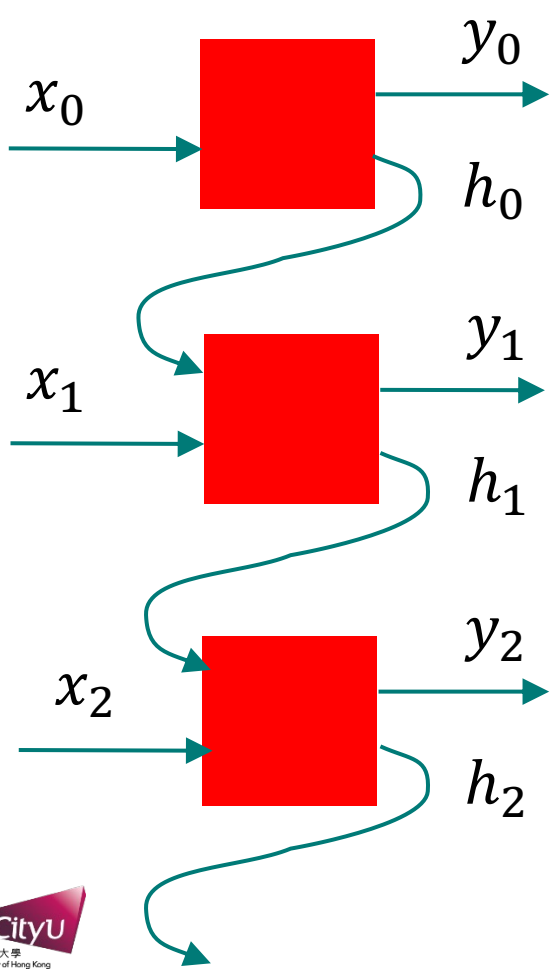
Directed Acyclic Graph

But its size depends on the input sequence length.

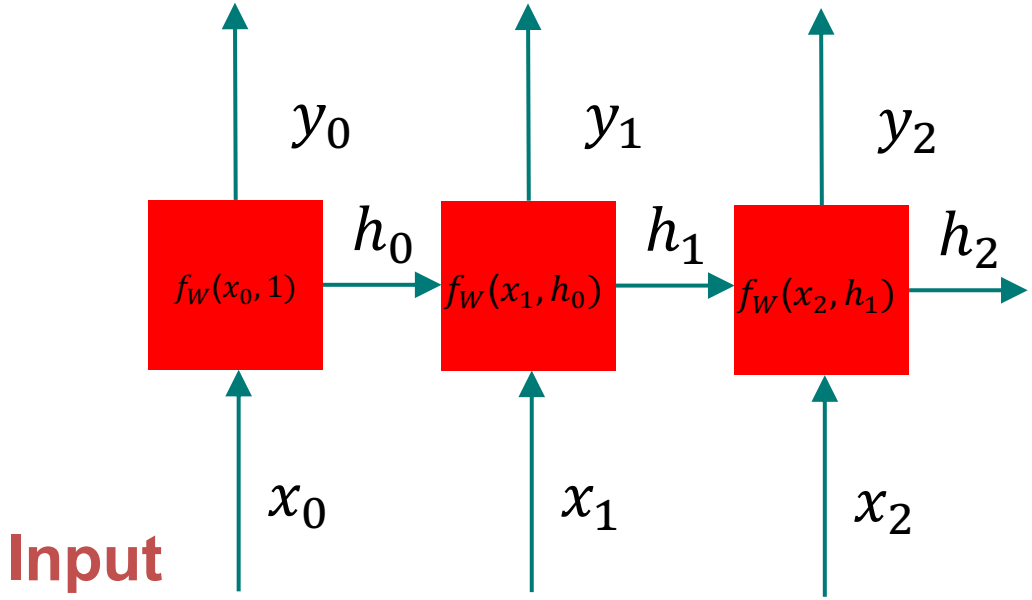
Repeated times

Unrolling RNNs

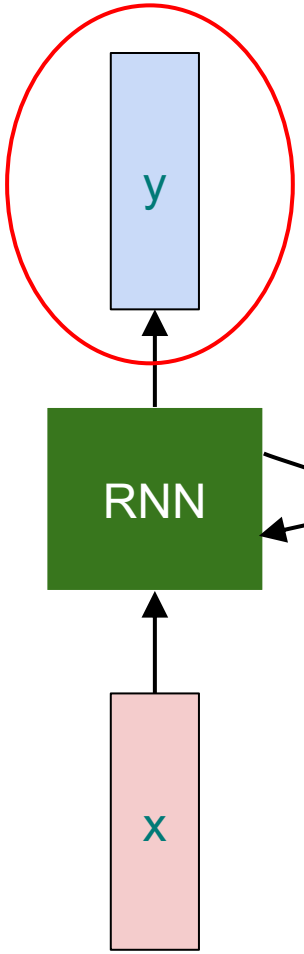
Usually drawn as:



Same function $f_W(\,)$: same parameters, same weights



RNN



Y: Target
(usually vector of a sequence)

h: Hidden State
(Information feedback to input layer)

X: Input
(Observations)

RNN for Time Series

We can process a sequence of vectors **x** by applying a recurrence formula at every time step:

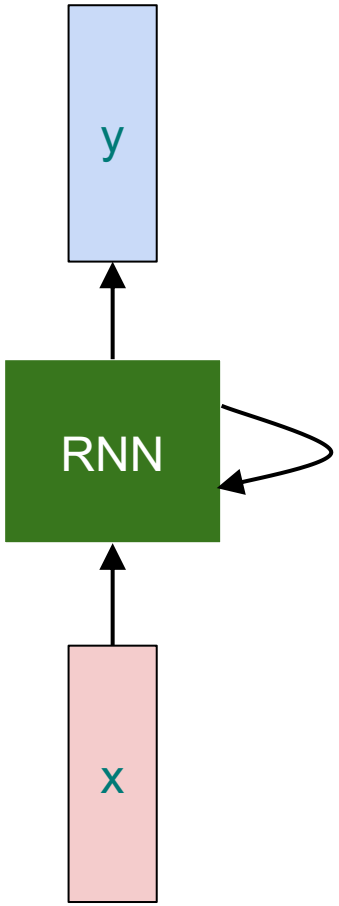
$$h_t = f_W(h_{t-1}, x_t)$$

new state

some function with parameters W

old state

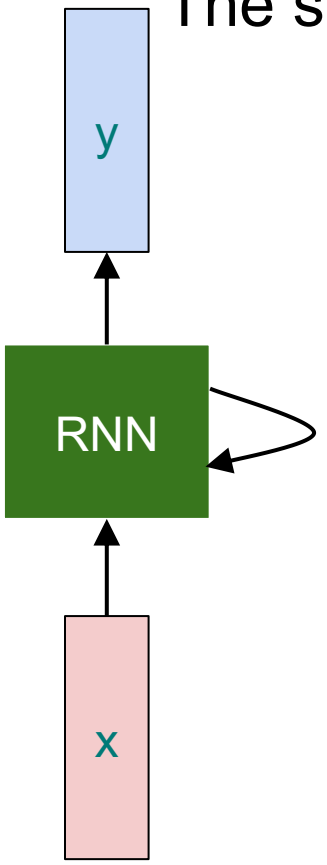
input vector at some time step



RNN Model is Simplified:
We have the same parameter set W for different time stamps t.

(Vanilla) RNN for Time Series

The state consists of a single *"hidden"* vector **h**:



$$h_t = f_W(h_{t-1}, x_t)$$

Tanh activation function

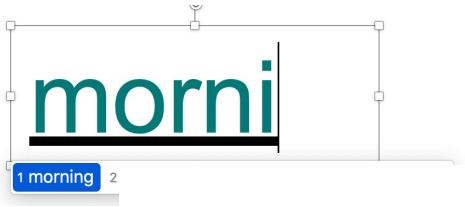
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

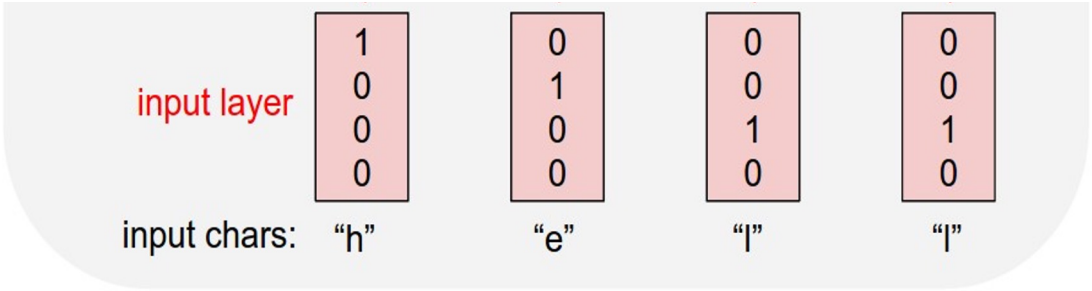
$$y_t = W_{hy}h_t$$

Example

Vocabulary:
[h,e,l,o]

Example training
Input sequence:
“hello”



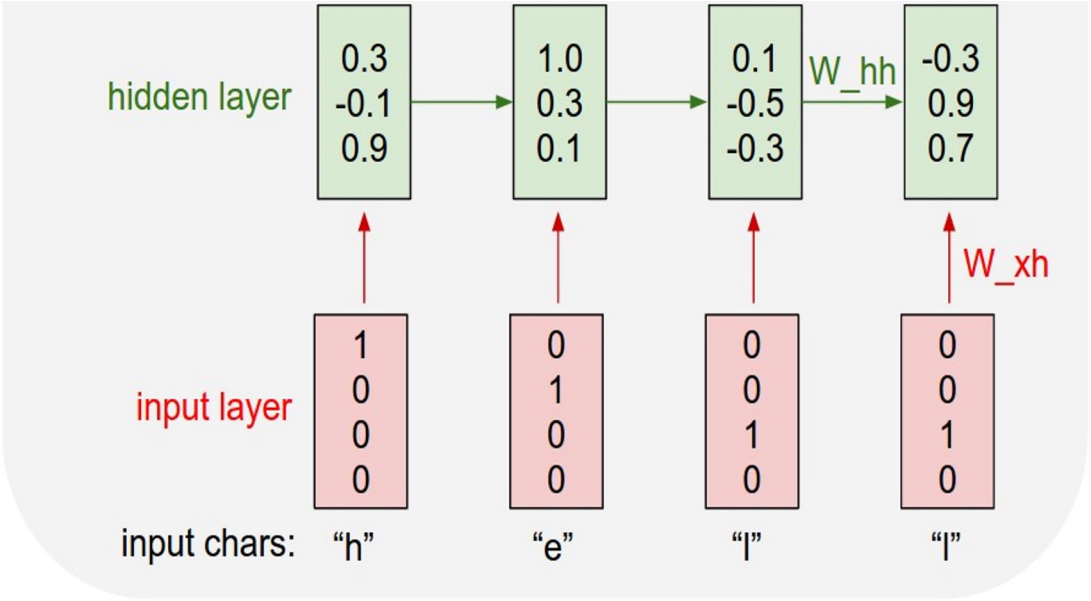


Example

Example hidden states calculation

(update hidden state)

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$



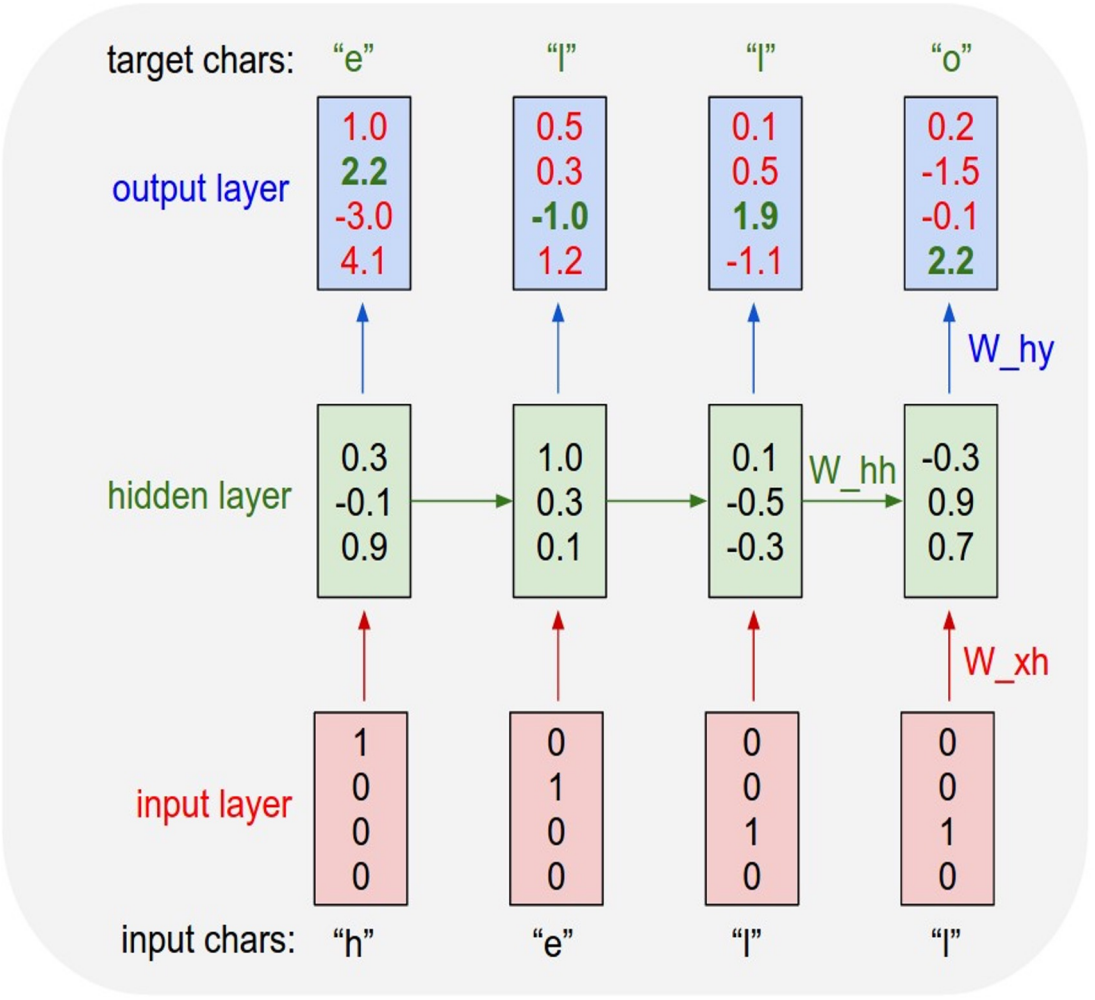
One-hot vector

Example

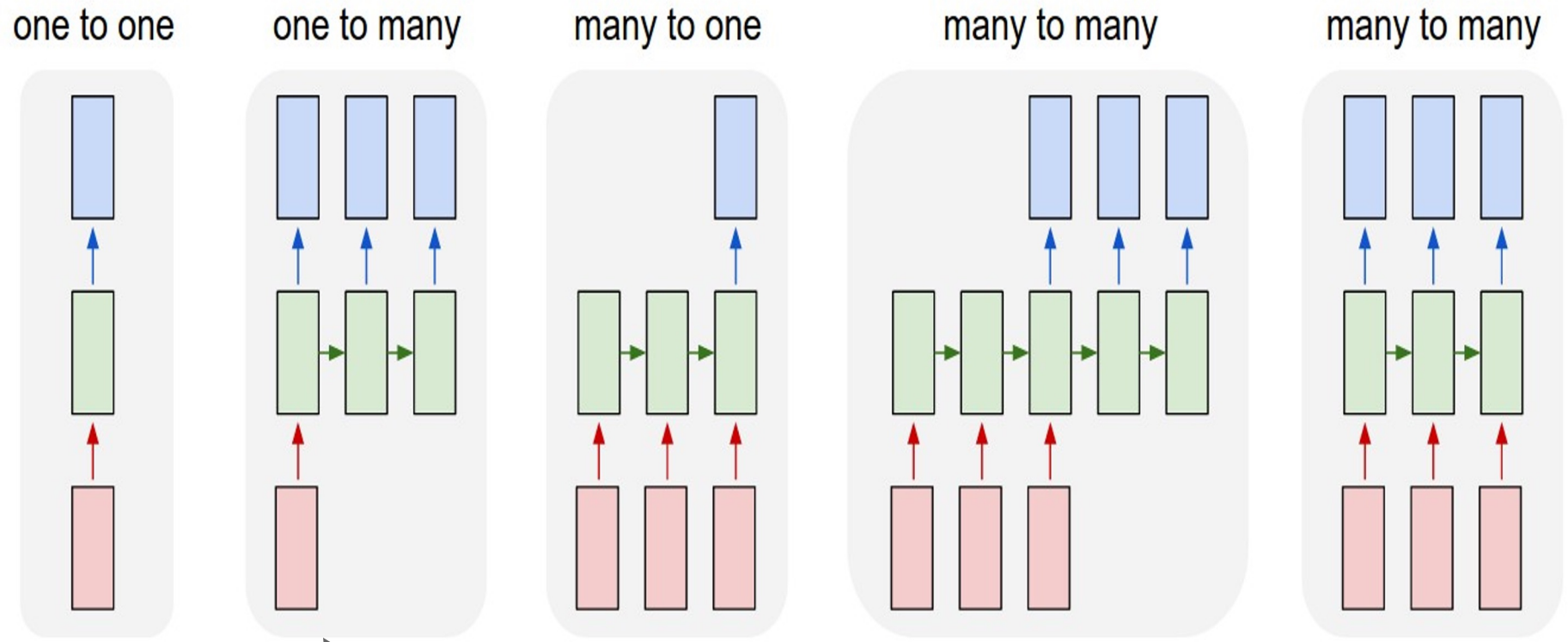
Example output vector calculation

$$y_t = W_{hy} h_t$$

Weights are shared

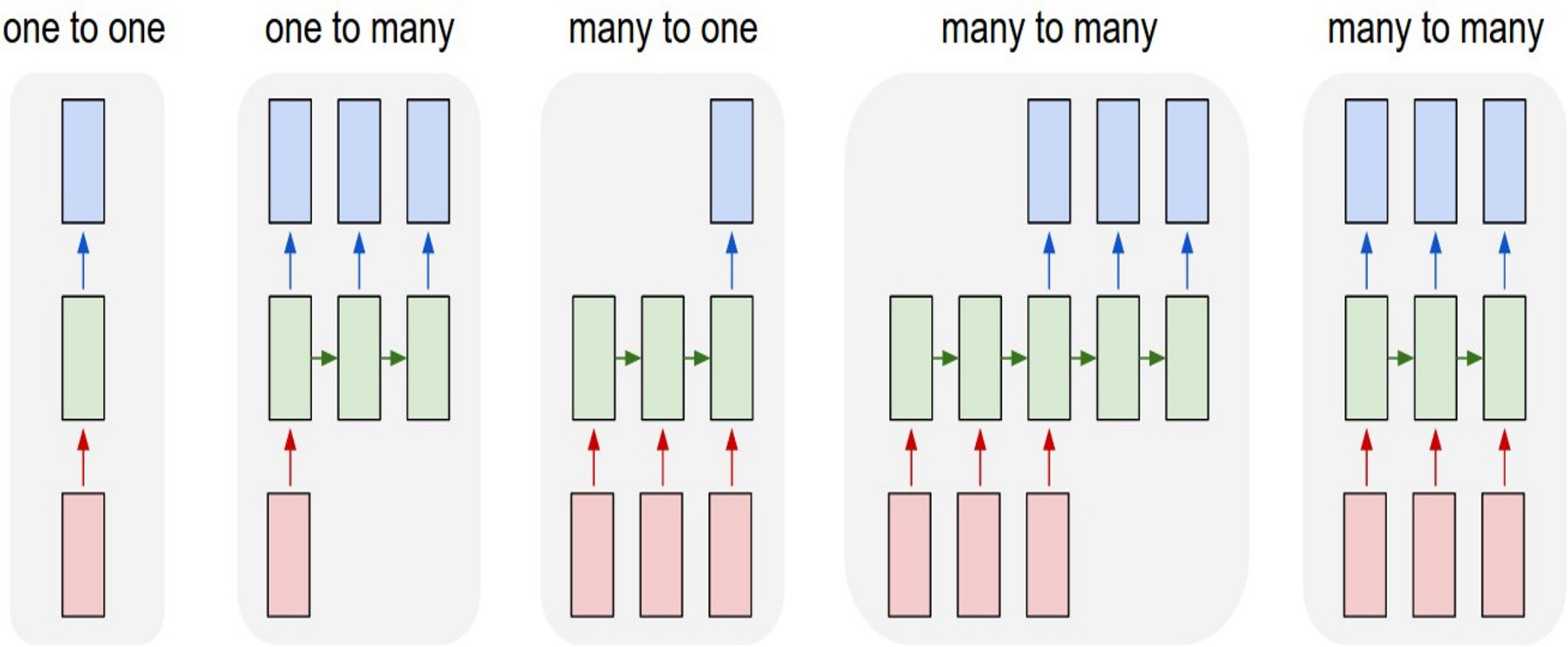


Recurrent Networks Offer Flexibility



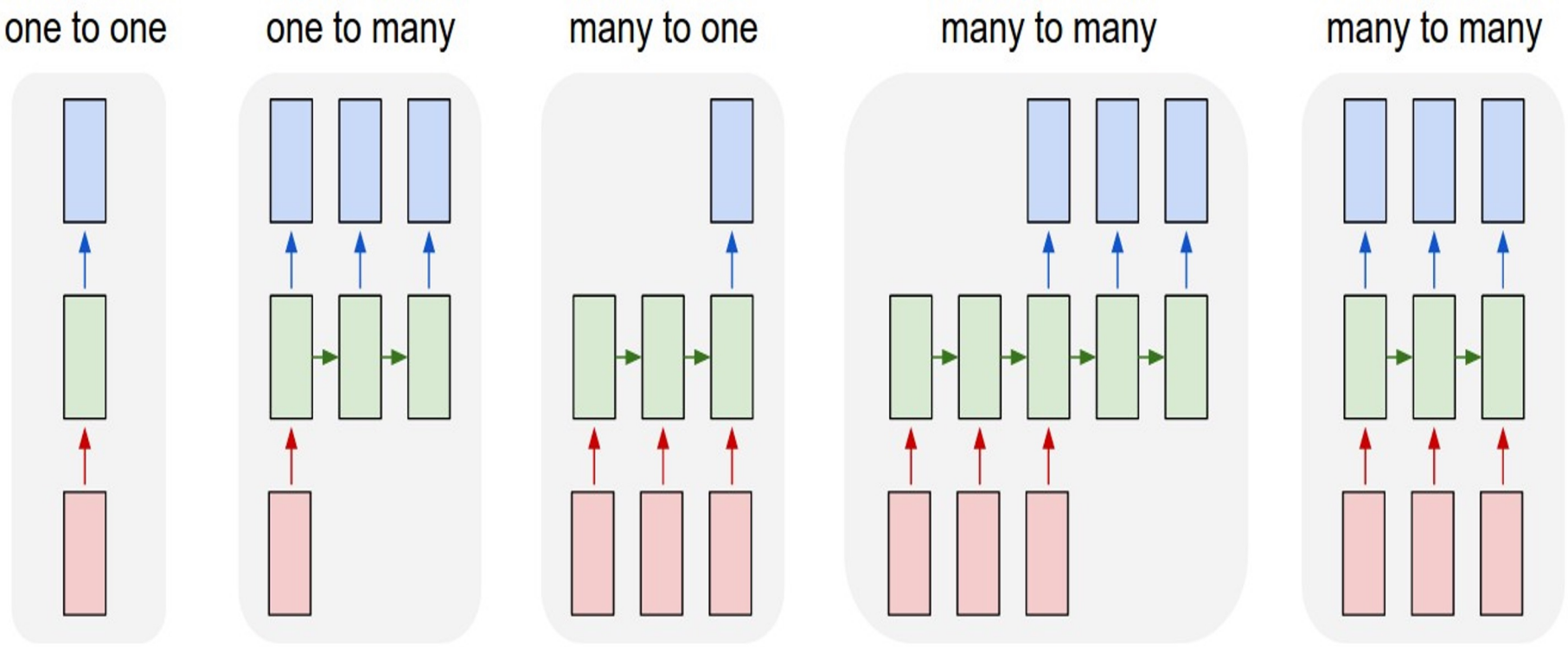
e.g. **Image Captioning**
image -> sequence of words

Recurrent Networks Offer Flexibility



e.g. **Sentiment Classification**
sequence of words -> sentiment

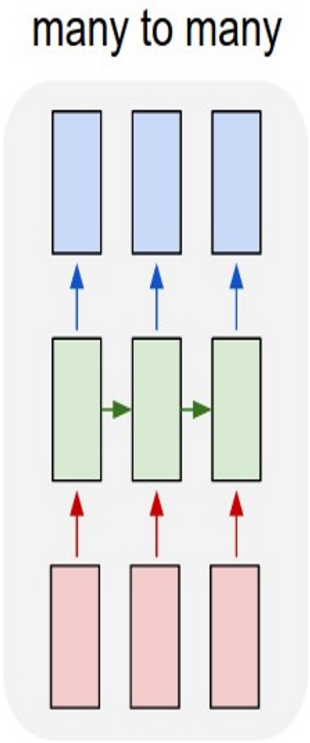
Recurrent Networks Offer Flexibility

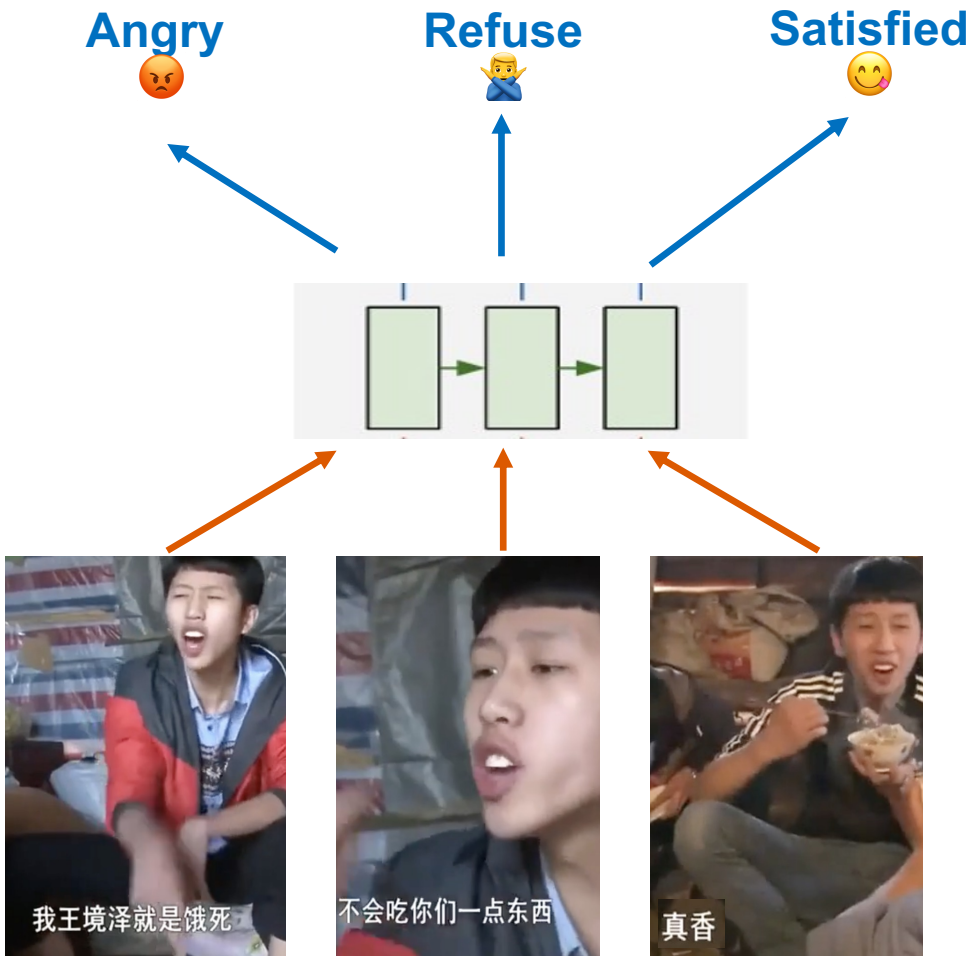


'Professor Liu's IS6400 course is so popular, it's hard to get a seat.'

e.g. **Machine Translation**
seq of words -> seq of words

Recurrent Networks Offer Flexibility



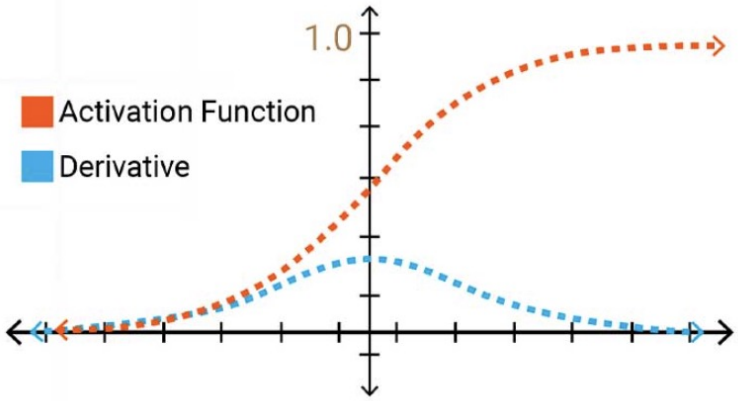


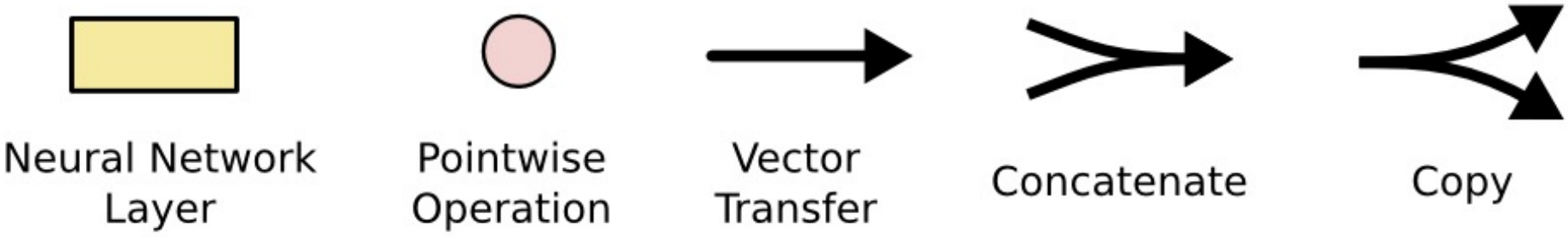
Video classification
Frame-by-frame Action Recognition

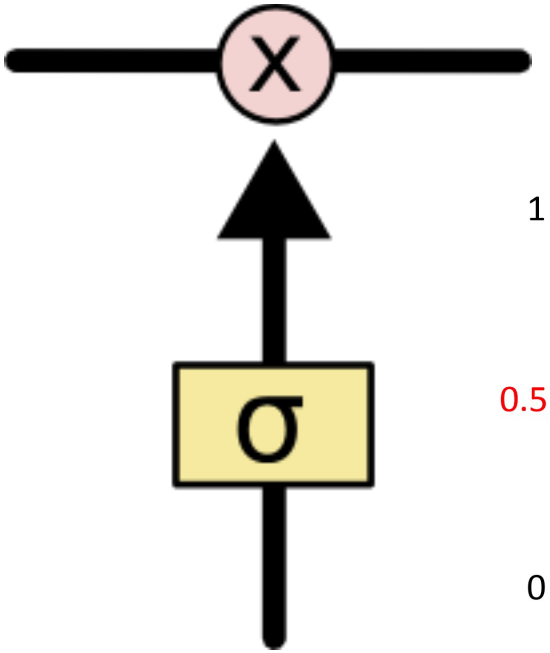
Problems of RNN

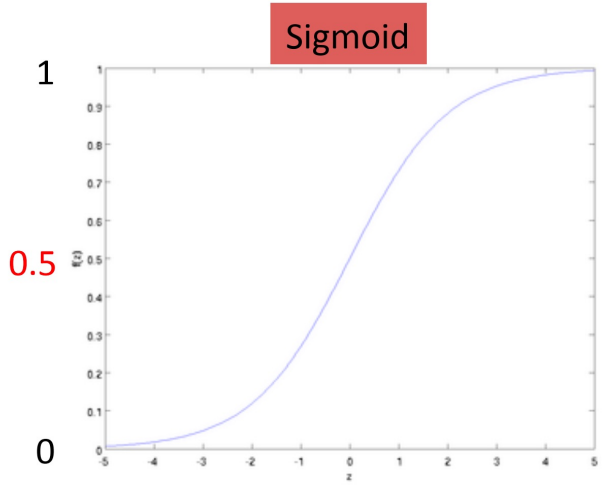
- When dealing with a time series, it tends to forget old information. When there is a distant relationship of unknown length, we wish to have a “memory” to it.
- Vanishing gradient problem.

Gradients from activations like sigmoid or tanh



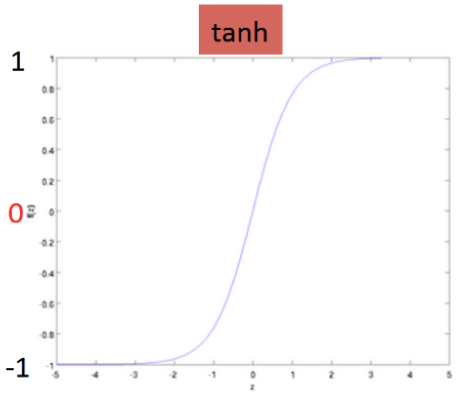


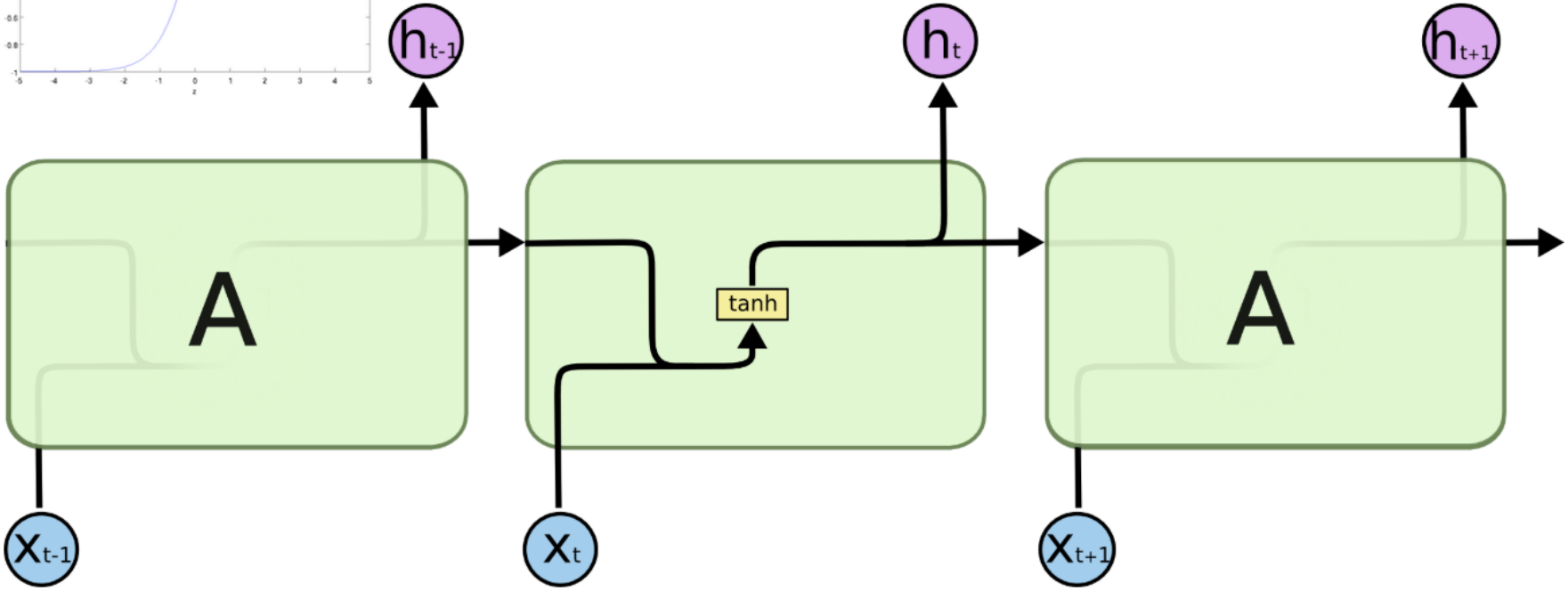




The sigmoid layer outputs numbers between 0-1 determine how much each component should be let through. Pink X gate is point-wise multiplication.

RNN Model

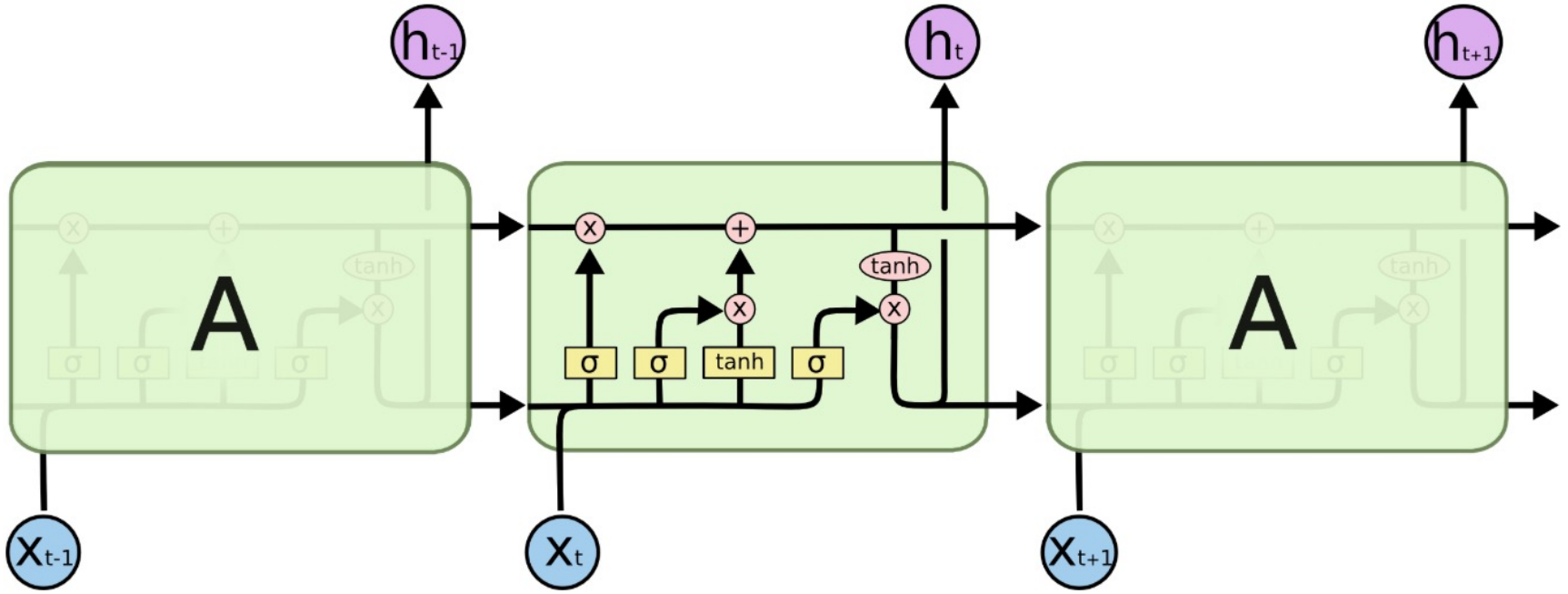




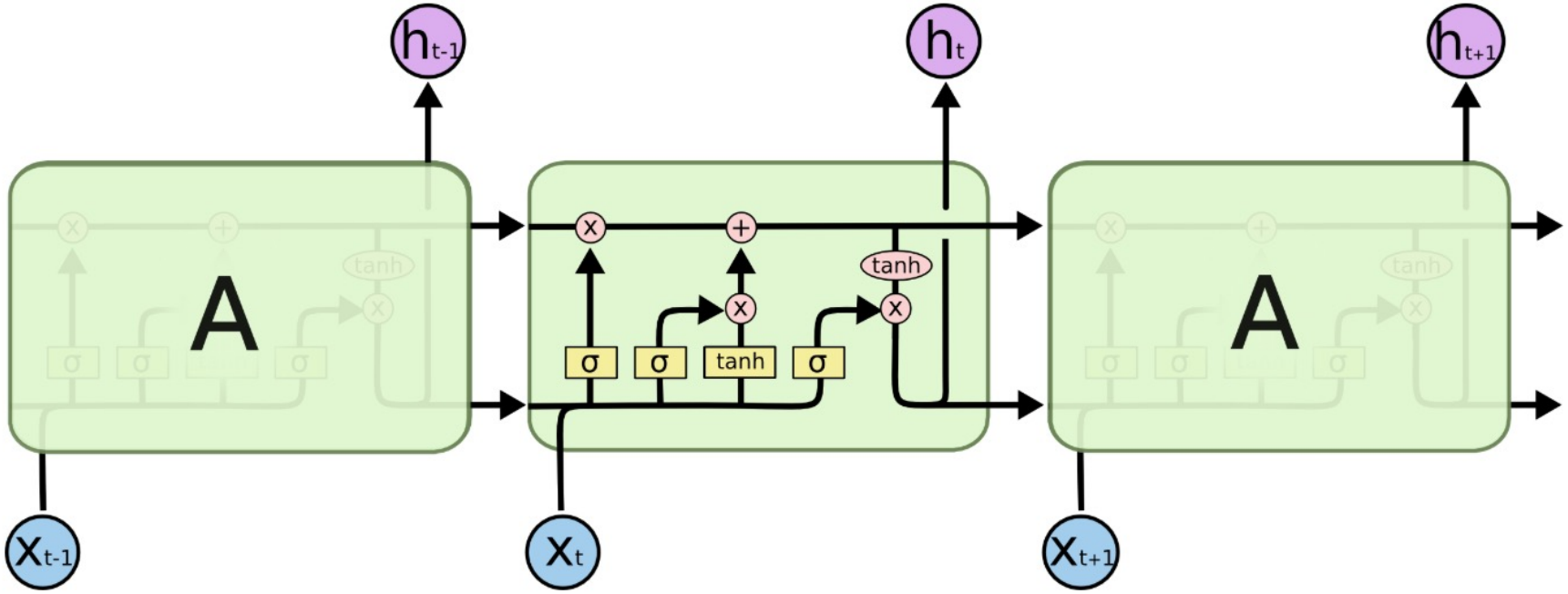
The repeating module in a standard RNN contains a single layer.

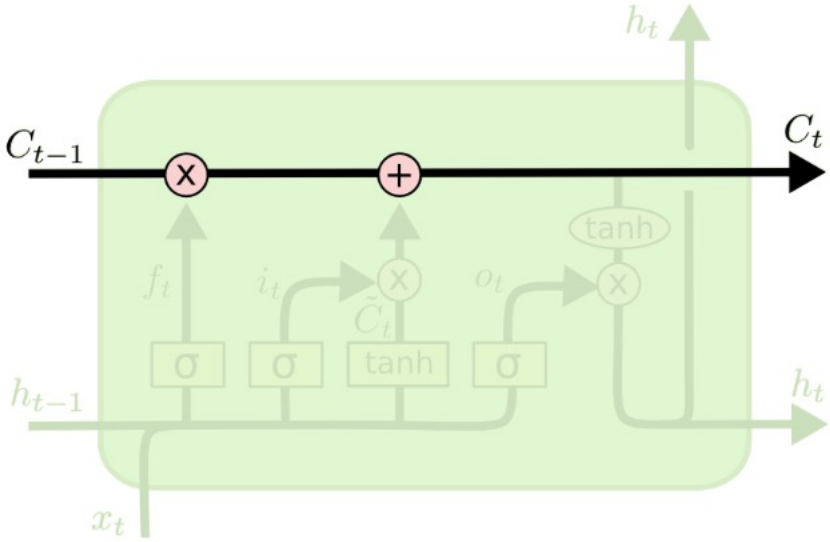
LSTM Model

LSTM: Long Term Short Term Memory



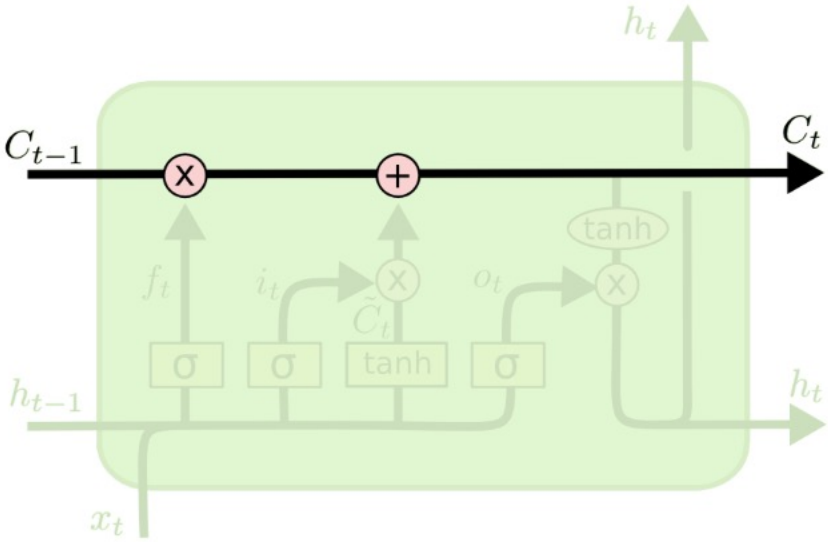
The repeating module in an LSTM contains four interacting layers.





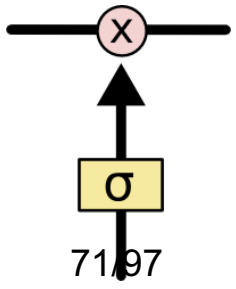
Cell State C:
like a conveyor belt connecting the entire structure

LSTM Cell State



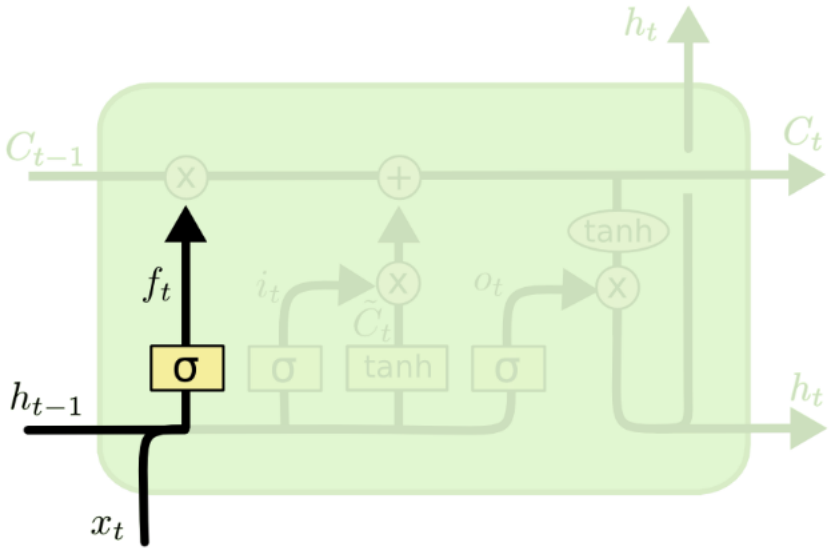
Cell State C:
like a conveyor belt connecting the entire structure

The LSTM does have the ability to remove or add long-term short-term information to the cell state, carefully regulated by structures called gates.



LSTM Walk-through

- 1. decide what information will be thrown away from the old cell state



Forget gate

Previous hidden state

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

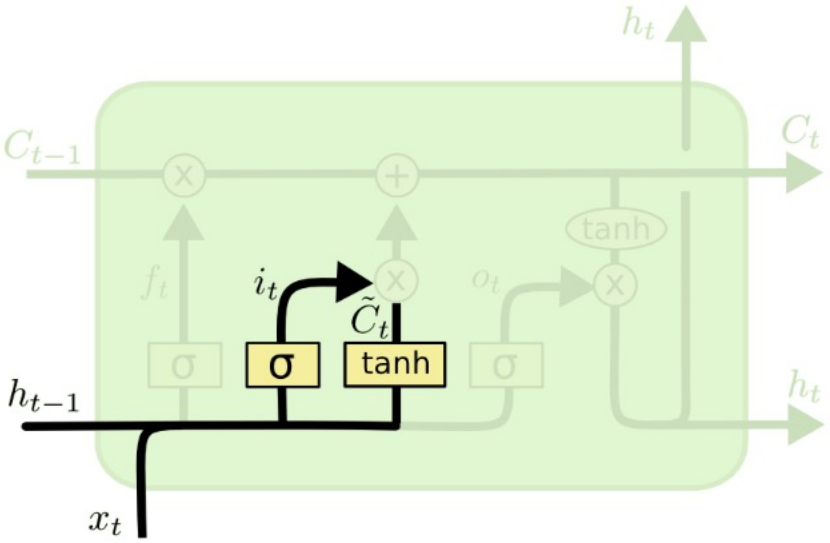
Current input

$f_t = 1$: keep this information

$f_t = 0$: throw this information

LSTM Walk-through

- 2. decide what new information will be generated and stored in current state.



Input gate

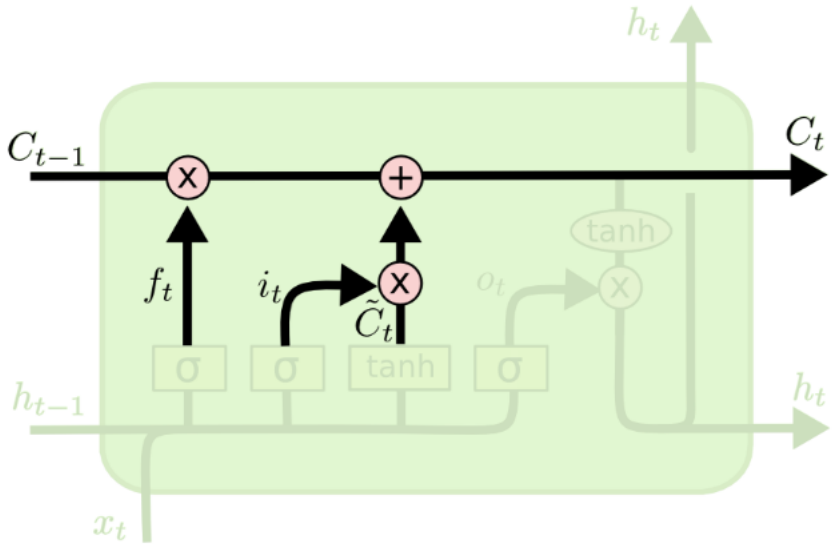
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Candidate value

LSTM Walk-through

- 3. Update old cell state Ct-1 to new cell state Ct (to be passed to next cell)



Candidate value

Input gate

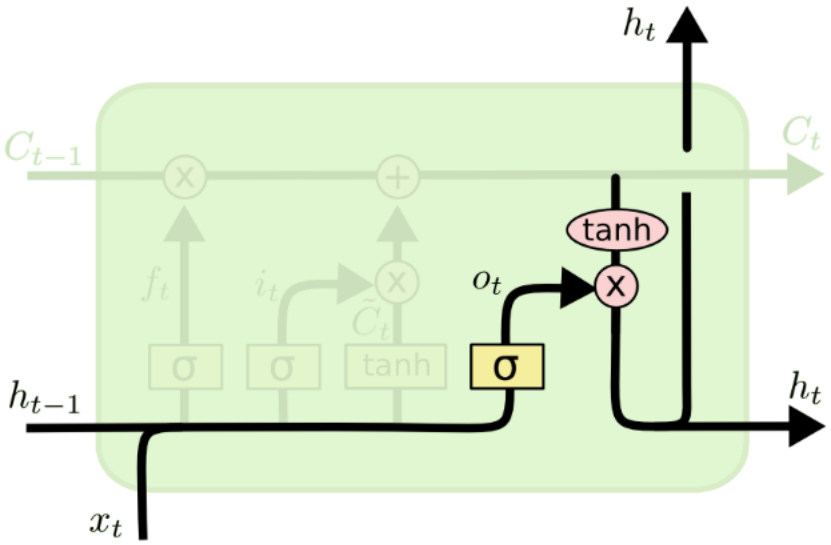
Forget gate

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Linear combination of old state and new created state information

LSTM Walk-through

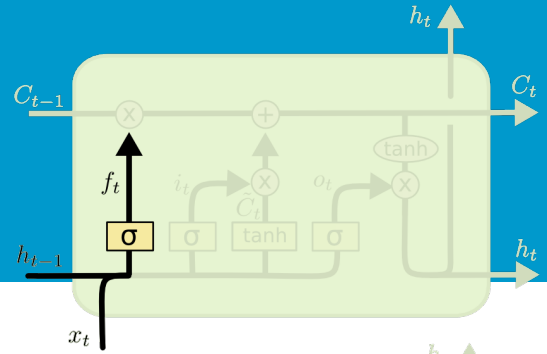
- 4. Generate output



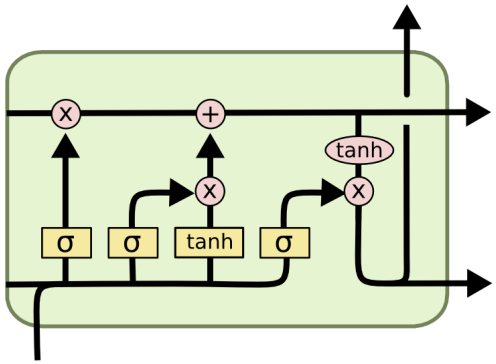
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

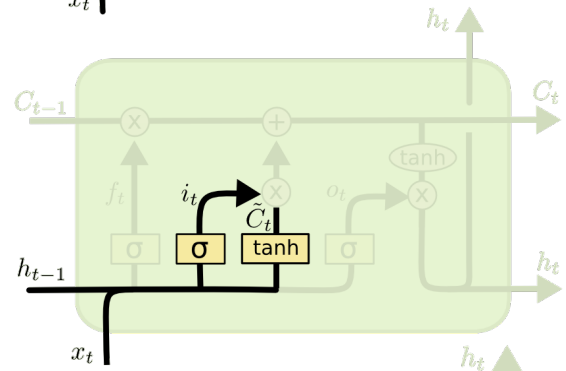
$$h_t = o_t * \tanh\left(C_t\right)$$

New cell state (updated)



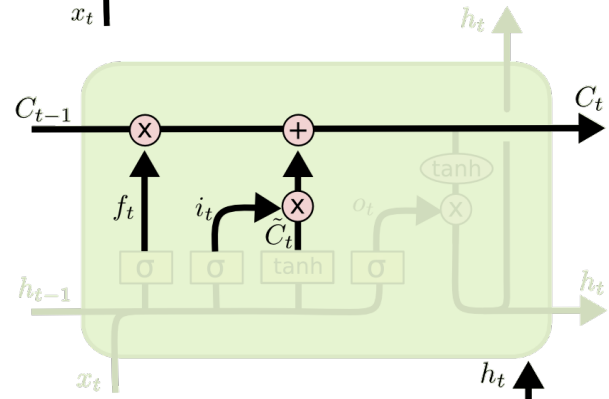
$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$





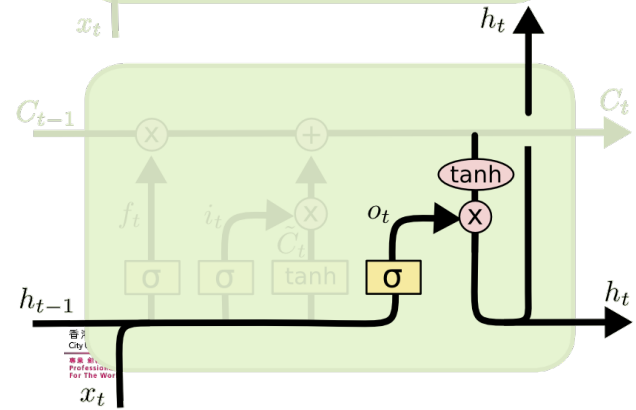
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

i_t decides what component is to be updated.
C'_t provides change contents



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Updating the cell state



$$o_t = \sigma\left(W_o \; [\, h_{t-1}, x_t] \; + \; b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Decide what part of the cell state to output

IS6400 Business Data Analytics

ZHANG Yanyu

City University of Hong Kong

Lecture 11

- ❑ Intro-Deep Learning
  - ❑ ANN
  - ❑ RNN, LSTM
  - ❑ NLP

Tutorial 11:

- ❑ Deep Learning

What is Text Mining

- Text mining is the set of techniques and methods used for the automatic processing of natural language text data available in reasonably large quantities in the form of computer files, …

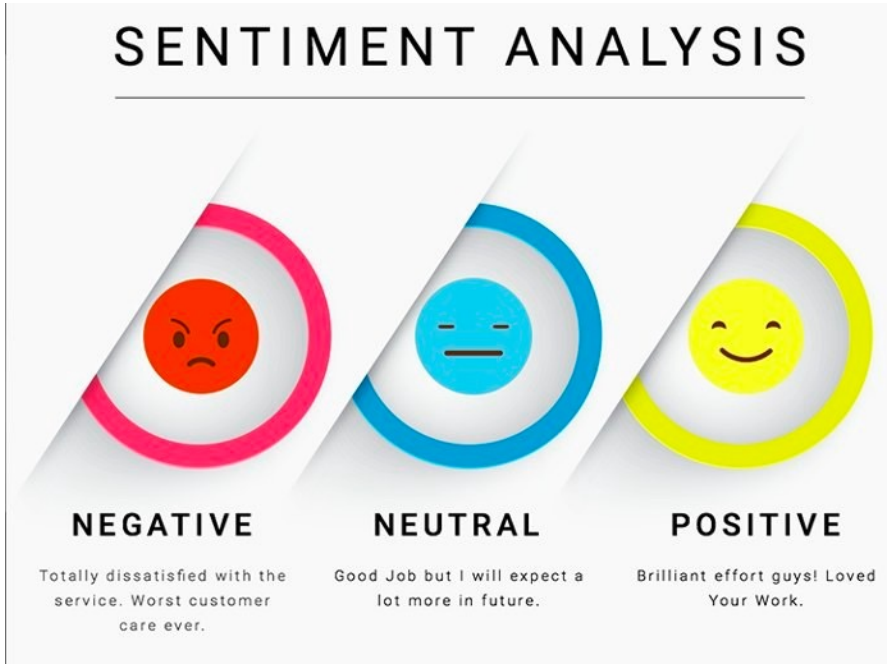
- with the aim of extracting and structuring their contents and themes, for the purposes of rapid (nonliterary) analysis, the discovery of hidden data, or automatic decision making.

Mining useful information from text data

- Same data mining tasks: clustering, classification, association analysis…
- Unstructured data: speech, document, social media articles, product review,…
- Text mining – first, impose structure to the data, then mine the structured data.

Examples of Text Mining 1

Sentiment analysis: a sequence of words -> sentiment



- Sentiment Analysis from Stanford University
- Tweet Sentiment Visualization from North Carolina State University:

https://www.csc2.ncsu.edu/faculty/healey/tweet_viz/tweet_app/

Examples of Text Mining 2

- Stock price forecasting
- **HedgeChatter – Social Media Stock Sentiment Analysis**



Examples of Text Mining 3

- Question – Answer (QA) System

Text Data:

COVID-19 is the infectious disease caused by the most recently discovered coronavirus. This new virus and disease were unknown before the outbreak began in Wuhan, China, in December 2019. The most common symptoms of COVID-19 are fever, tiredness, and dry cough. Some patients may have aches and pains, nasal congestion, runny nose, sore throat or diarrhea. These symptoms are usually mild and begin gradually. Some people become infected but don't develop any symptoms and don't feel unwell. Most people (about 80%) recover from the disease without needing special treatment. Around 1 out of every 6 people who gets COVID-19 becomes seriously ill and develops difficulty breathing. Older people, and those with underlying medical problems like high blood pressure, heart problems or diabetes, are more likely to develop serious illness. People with fever, cough and difficulty breathing should seek medical attention.

QA system

Question 1 *

What is COVID-19?

the infectious disease caused by the most recently discovered coronavirus

Question 2

What are symptoms of COVID-19?

fever, tiredness, and dry cough

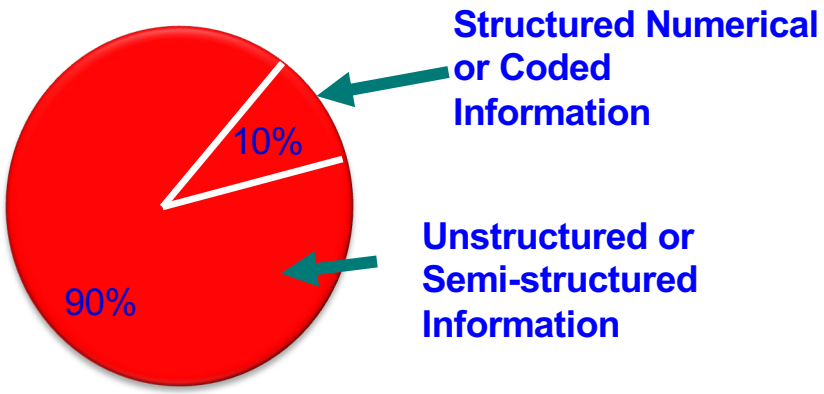
Question 3

How many people recover from COVID?

about 80%

Why Text Analytics/Mining ?

- Approximately **90%** of the world's data is held in unstructured formats (source: Oracle Corporation)
- Information intensive business processes demand that we transcend from simple document retrieval to "knowledge" discovery.



Text Analytics/Mining Methods (1)

- Information Retrieval Style: Information Search
  - *Search technology* (e.g. representation, indexing, ranking)
  - *TF-IDF* (Term Frequency –Inverse Document Frequency)
- Natural Language Processing Style: Information Extraction
  - *Entity extraction:* e.g. person names, organizations, locations, regulation codes)
  - *Relation extraction:* e.g. extracted relationships usually occur between two or more entities of a certain type such as person) and fall into a number of semantic categories such as *married to*, *employed by*, *live in*.
  - *Syntax (e.g. grammars, parsers, parse trees)*, *semantics and ontology (e.g. semantic classes, subcategorization frames)*
  - *Contextual analysis*

Challenges of Text Analytics

- Very high number of possible “dimensions”
  - All possible word and phrase types in the language!!
- Unlike data mining:
  - records (= docs) are not structurally identical
  - records are not statistically independent
- Complex and subtle relationships between concepts in text
  - “AOL merges with Time-Warner”
  - “Time-Warner is bought by AOL”
- Ambiguity and context sensitivity
  - automobile = car = vehicle = Toyota
  - Apple (the company) or apple (the fruit)

How to represent a document?

- A document representation aims to capture what the document is about
- Main approach:
  - Each entry in the table represents a document
  - Attributes represent the frequency in which a term appears in the document

Example: Document-Term frequency table/matrices

Documents

	Terms				
	Camera	Digital	Memory	Print	…
Document 1	3	2	0	1	
Document 2	0	4	0	3	
…	…	…	…	…	

Some Basic Information Retrieval Techniques

- Stemming: cut words to their root.
  - Taxes, taxation (both have the root “tax”)
- Stop words: irrelevant words for removal.
  - E.g. if, and, but, who, what, the, they, their, a, or…
- Weighting of terms (e.g., Term frequency–Inverse document frequency (TF-IDF))
  - *Corpus*: a collect of documents
  - An approach for weighting terms in a document based on the term’s frequency in the document and the document corpus.
  - A term would have a higher weight if it is found to be a good descriptor for a document, i.e., if it appears frequently in the document but is infrequent in the entire corpus.

Term Frequency (TF)

Term Frequency (TF): the frequency of a word appears in a document.

- Let $Count(i,j)$ be the count of the *term i* in the *document j*.
- Let $TF(i,j)$ be the proportion of the count of *term i* in the *document j.*

$$TF(i,j) = \frac{count(i,j)}{Total\ number\ of\ words\ in\ document\ j}$$

Note: the total number of words is used to normalize the direct counting of Count(i,j). If one document has much more words then the other, we need to normalize the Count(i,j) based on document size.

The size of a document, can also be replaced by the total frequency of terms (keywords) in this document.

Inverse Document Frequency (IDF)

- ***Inverse document frequency***: the number of documents which contain a specific word.
- Let *N* be the count of distinct documents in the corpus.
- Let *Count (i)* be the count of documents in the corpus in which the term *i* is present.
- The **inverse document frequency** is a measure of how much information the word provides, i.e., if it's common or rare across all documents.

$$IDF(i) = \log(\frac{N}{count(i)})$$

The Mighty TF-IDF

- Combine TF and IDF to enhance document-term frequency matrices:

$$\text{TF\&IDF}(i,j) = TF(i,j) * IDF(i)$$

attribute

	Terms				
	Camera	Digital	Memory	Print	…
Document 1	3	2	0	1	
Document 2	0	4	0	3	
…	…	…	…	…	



Clustering
Classification
Topic models
Regression

What is Natural Language Processing (NLP)?

- In 1950, Alan Turing Published an article titled “Machine and Intelligence” which advertised what is now called the Turing test as a subfield of intelligence.
- Process information contained in natural language text
- How to program computers to process and analyze large amounts of natural language data.
  - speech recognition
  - natural language understanding
  - natural language generation.



(source: Data Science Central)

Fundamental Tasks

- Word Segmentation
- Part-of-speech (POS) tagging
- Syntactic Analysis
- Semantic Analysis

Word Segmentation

Separate a sentence into segments

Michael Jordan is a professor at Berkeley



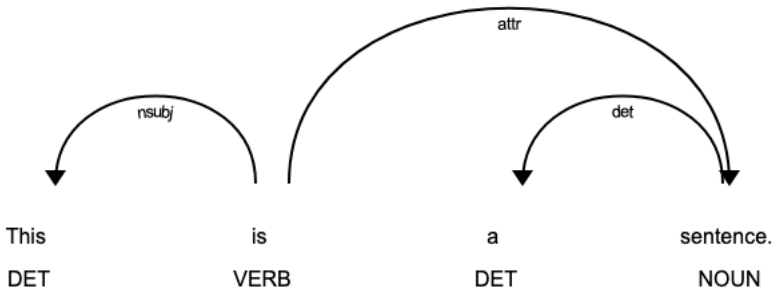


浦东开发与建设 → 浦东 / 开发 / 与 / 建设

Pudong development and construction

POS tagging

- Each word in a sentence can be classified in to classes, such as verbs, adjectives, nouns, etc
- POS Tagging is a process of tagging words in a sentences to part-of-speech, based on:
  - – Its definition
  - – Its context in the sentence
- Can be achieved by sequential labeling

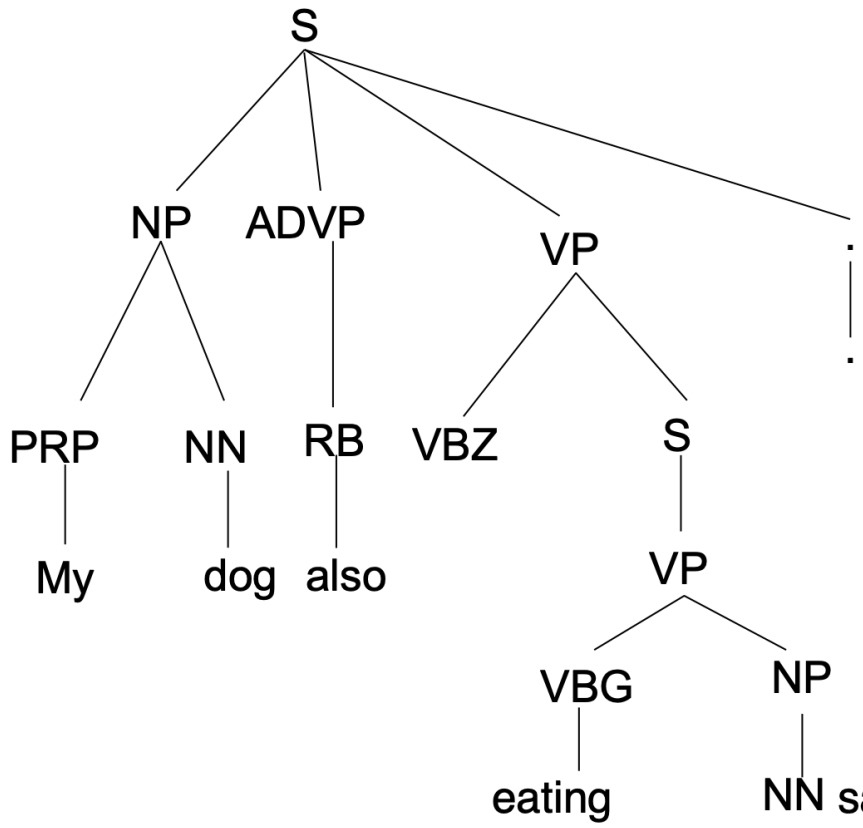


Syntactic Analysis

- The task of recognizing a sentence and assigning a syntactic structure to it.

My dog also likes eating sausage.

```
(ROOT
  (S
    (NP (PRP$ My) (NN dog))
    (ADVP (RB also))
    (VP (VBZ likes)
      (S
        (VP (VBG eating)
          (NP (NN sausage)))))
    (. .)))
```



Semantic Analysis

- Two levels
  - Lexical semantics − Representing meaning of words (e.g., word bank)
  - Compositional semantics − How words combined to form a larger meaning.

‘Colorless green ideas sleep furiously’
grammatically correct, but semantically nonsense.

Process of NLP

Morphological and Lexical Processing

- Analyze, identify and describe of structure of words.
- Divide a text into paragraphs, words and the sentences.

Syntactic Analysis

- Concerns the proper ordering of words and its effect on meaning.
- Involves analysis of the words in a sentence to describe grammatical structure of the sentence. E.g. "the girl the go to the school". This would definitely be rejected by the English syntactic analyzer.

Semantic Analysis

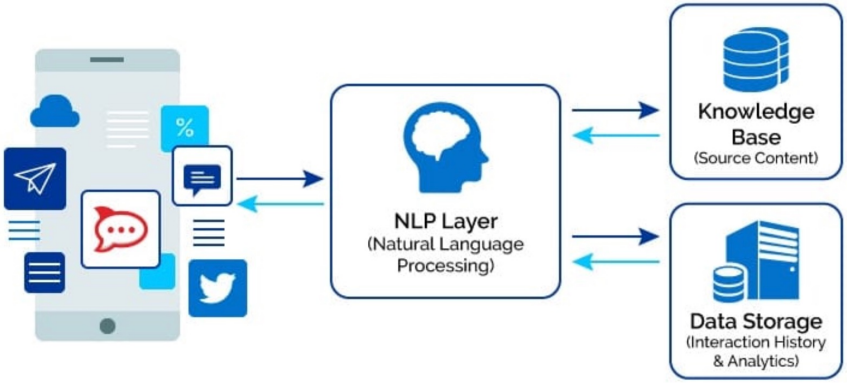
- Concerns the literal meaning of words, phrases, and sentences.
- The structures which are created by the syntactic analyzer are assigned meaning. E.g. "colorless blue idea". This would be rejected by the analyzer.

Context Processing Interpretation

- Sense of context
- The meaning of any single sentence depends on the sentences that precedes it and also invokes the meaning of the sentences that follow it. e.g. the word "it" in the sentence "she wanted it" depends on the prior discourse context.

Some NLP Platforms

- Dialog Flow (ex API.ai)
- Wit.ai
- Luis
- Lex
- Watson Assistant



IS6400 Business Data Analytics

ZHANG Yanyu

City University of Hong Kong

Lecture 11

- ❑ Intro-Deep Learning
  - ❑ ANN
  - ❑ RNN, LSTM
  - ❑ NLP

Tutorial 11:

- ❑ Deep Learning